TABLE OF CONTENTS

CHAPTER NO:	TITLE	PAGE NO:
	ABSTRACT	i
	LIST OF TABLES	ii
	LIST OF FIGURES	iii
	LIST OF ABBREVIATIONS	iv
1	**INTRODUCTION**	1
	1.1 OVERVIEW OF THE PROJECT	2
	1.2 LITERATURE SURVEY	2
	1.2.1 AI Based Healthcare and Management System	2
	1.2.2 Intelligent Disease Prediction and OTC Recommendation System	2
	1.2.3 AI Powered Medical Diagnosis Support Systems	2
	1.2.4 Digital Health Consultation and Monitoring Systems	2
	1.2.5 AI Driven Mental Health Assessment and Prediction System	2
	1.3 PROPOSED SYSTEM OBJECTIVES AND SCOPES	2
	1.3.1 Objectives of the Proposed system	2

CHAPTER NO:	TITLE	PAGE NO:
	1.3.2 Scope of the Proposed system	2
2	**REQUIREMENTS SPECIFICATIONS**	3
	2.1 OVERALL DESCRIPTION	3
	2.1.1 Functional Requirements	4
	2.2 PRODUCT PERSPECTIVE	4
	2.3 PRODUCT FUNCTIONS	4
	2.3.1 Disease Prediction Configuration	5
	2.3.2 AI Based Disease Prediction	5
	2.3.3 Multimodal Health Data Submission	5
	2.3.4 Real-time Health Analysis	5
	2.3.5 Intelligent OTC Medicine Recommendation	5
	2.3.6 Healthcare Reporting and PDF Generation	5
	2.3.7 Geolocation Hospital Routing	5
	2.3.8 User Account and Prediction History Management	5
	2.3.9 Admin Monitoring and Inventory Management	5
	2.4 USER CHARACTERISTICS	5

CHAPTER NO:	TITLE	PAGE NO:
	2.4.1 Admin	5
	2.4.2 User (General User / Patient)	5
	2.5 OPERATING ENVIRONMENT CONSTRAINTS	5
	2.5.1 Software Specification	5
	2.5.2 Hardware Specification	5
	2.6 SPECIFIC REQUIREMENTS	5
	2.6.1 User Requirements	5
	2.6.2 Admin Requirements	5
	2.6.3 Functional Requirements	6
	2.6.4 Non-Functional Requirements	6
	2.6.5 Security Requirements	6
	2.7 EXTERNAL INTERFACE REQUIREMENTS	6
	2.8 SYSTEM FEATURES	6
3	**SYSTEM DESIGN AND TEST PLAN**	7
	3.1 DATA FLOW DIAGRAM (DFD)	7
	3.1.1 Level 0 DFD	8-9
	3.1.2 Level 1 DFD	9-10

CHAPTER NO:	TITLE	PAGE NO:
	3.2 USE CASE DIAGRAM	11
	3.2.1 Admin Use Case Diagram	11-12
	3.2.2 User Use Case Diagram	12-14
	3.3 TABLE DESIGNS	15
	3.3.1 Users Collection (users)	15
	3.3.2 Prediction History Collection (prediction_history)	16
	3.3.3 Medicines Collection (medicines)	17
	3.3.4 Diseases Collection (diseases)	18
	3.3.5 System Activity Logs Collection (activity_logs)	19-20
	3.4 PERFORMANCE REQUIREMENTS	21
	3.5 SOFTWARE QUALITY ATTRIBUTES	22
	3.6 DETAILED DESIGN	23
	3.6.1 System Architecture	23
	3.6.2 User Registration and Authentication	23
	3.6.3 User Profile and Management	24
	3.6.4 AI-Based Disease Prediction and OTC Recommendation	24

CHAPTER NO:	TITLE	PAGE NO:
	3.6.5 Symptom Data Processing and Evaluation	25
	3.6.6 PDF Reporting and Administrative Analytics	25
	3.6.7 Security Measures	25
	3.6.8 Performance Optimization	26
	3.6.9 Testing and Deployment	26
	3.7 TEST CASES	26
	3.7.1 Test Cases for Admin	26-28
	3.7.2 Test Cases for User	29-30
4	**IMPLEMENTATION AND RESULTS**	31
	4.1 IMPLEMENTATION	31
	4.1.1 Backend Development	31
	4.1.2 Frontend Development	32
	4.1.3 Integration and Testing	32
	4.2 IMPLEMENTATION RESULTS	33
5	**CONCLUSION AND FUTURE WORKS**	34
	5.1 CONCLUSION	34
	5.2 FUTURE WORKS	35

CHAPTER NO:	TITLE	PAGE NO:
	5.2.1 Future Implementation Plan	36
	REFERENCES	37
	APPENDIXES	37
	Appendix A- Screenshots	37-41
	Appendix B- Sample Code	42-59

ABSTRACT

The rapid and unchecked proliferation of medical data on the internet has led to a significant rise in unverified self-diagnosis and unsupervised self-medication, posing severe health risks and delaying critical clinical interventions. To mitigate this issue, this project presents the **AI-Powered Medical Assistant & OTC Recommendation System**, an intelligent, secure, and cloud-hosted web platform designed to bridge the gap between initial symptom onset and professional medical care.

The core objective of the system is to provide a reliable preliminary triage tool that helps users identify possible minor illnesses and receive general information regarding verified Over-The-Counter (OTC) medicines. The platform leverages supervised Machine Learning—specifically a **Random Forest Classifier** trained on comprehensive symptom-to-disease datasets—to analyze user-inputted symptoms and accurately predict potential health conditions. A key technological innovation of this system is the integration of a custom **Relative Confidence Margin** algorithm (Top_Prob / (Top_Prob + RunnerUp_Prob)), which mathematically evaluates the decisiveness of the model against competing classes, providing transparent and realistic confidence metrics to the user.

To prioritize clinical safety above all, the recommendation engine enforces a strict algorithmic filter that recommends *only* verified Over-The-Counter (OTC) medications while programmatically excluding prescription drugs, controlled substances, and antibiotics. The system architecture is engineered using **Python** and the **Flask** framework (employing modular MVC routing via Blueprints) coupled with a highly scalable **MongoDB Atlas** cloud database. Enterprise-grade security is established through **Werkzeug PBKDF2** password hashing, a secure 'Forgot Password' workflow, and isolated Role-Based Access Control (RBAC) verified against environment variables for administrative operations.

Furthermore, the platform incorporates advanced functional modules including an interactive Admin Dashboard with full **CRUD operations** (Create, Read, Update, Delete) for dynamically managing diseases, precautions, and medicine inventories. It also features session history tracking, automated **PDF report generation** via ReportLab utilities, and real-time **HTML5 Geolocation routing** to instantly connect users with nearby healthcare facilities during emergencies. By combining machine learning predictive accuracy with robust safety constraints and modern web technologies, the system serves as an exemplary educational and preliminary healthcare support platform.

LIST OF TABLES

TABLE NO:	TITLE	PAGE NO:
3.1	Users Collection (users)	33
3.2	Prediction History Collection (prediction_history)	33-34
3.3	Medicines Collection (medicines)	34-35
3.4	Diseases Collection (diseases)	35
3.5	System Activity Logs Collection (activity_logs)	36

LIST OF FIGURES

FIGURE NO:	TITLE	PAGE NO:
3.1	DFD Level 0	28
3.2	DFD Level 1	29
3.3	Admin Use Case Diagram	31
3.4	User Use Case Diagram	32
A.1	Home Page	63
A.2	Login & Registration Page	63
A.3	Admin Control Panel & Dashboard	64
A.4	User Dashboard & Symptom Input Interface	64
A.5	AI Disease Prediction & OTC Medicine Results	65
A.6	Downloadable PDF Medical Triage Report	65

LIST OF ABBREVIATIONS

SL NO:	ABBREVIATIONS	MEANING
1	AI	Artificial Intelligence
2	API	Application Programming Interface
3	CSS	Cascading Style Sheets
4	DFD	Data Flow Diagram
5	HTML	Hypertext Markup Language
6	HTTP	Hypertext Transfer Protocol
7	ML	Machine Learning
8	NoSQL	Non-Structured Query Language (MongoDB Atlas)
9	OTC	Over-The-Counter (Medicines)
10	PDF	Portable Document Format
11	RF	Random Forest
12	UI / UX	User Interface / User Experience

CHAPTER 1

INTRODUCTION

In today's rapidly evolving digital healthcare environment, the demand for intelligent and safe medical support systems has increased significantly due to the growing volume of unverified online health information, limited clinical resources, and the widespread risk of unsupervised self-medication. Traditional internet-based health searches often lack structured safety boundaries, leading users toward inappropriate self-diagnosis, panic, or dangerous consumption of prescription drugs. To address these critical issues, the **AI-Powered Medical Assistant & OTC Recommendation System** has been developed as an intelligent web-based platform that integrates machine learning, strict clinical safety filtering, and modern web technologies into a unified ecosystem.

The proposed platform is designed to provide users with a secure preliminary health triage environment where individuals can input symptoms and receive reliable, non-prescription guidance. The system utilizes supervised machine learning algorithms—specifically a **Random Forest Classifier**—to analyze complex symptom patterns and predict potential minor illnesses. A key innovation of this platform is the implementation of a custom **Relative Confidence Margin** formula (Top_Prob / (Top_Prob + RunnerUp_Prob)), which mathematically normalizes raw multi-class probabilities to accurately reflect the AI's decisiveness and provide transparent metrics to the user.

In addition to intelligent symptom prediction, the platform includes comprehensive management functionalities such as secure user authentication, prediction history tracking, downloadable PDF medical reports via ReportLab utilities, and real-time **HTML5 Geolocation routing** to connect users with nearby healthcare facilities during emergencies. To prioritize clinical safety above all, the system enforces a strict algorithmic filter that recommends *only* verified Over-The-Counter (OTC) medications while programmatically excluding prescription drugs, narcotics, and antibiotics. Furthermore, an administrative module enables authorized personnel to manage diseases, precautions, and medicine inventories dynamically through a fully functional CRUD interface.

The system is developed using **Python** and the **Flask** framework (utilizing modular MVC routing via Blueprints) paired with a highly scalable **MongoDB Atlas** cloud database. Enterprise-grade security is established through **Werkzeug PBKDF2** password hashing and isolated Role-Based Access Control (RBAC) verified against environment variables. By combining intelligent prediction systems, strict safety constraints, and modern web development, the project serves as an advanced, educational, and responsible preliminary healthcare support platform.

1.1 Overview of the Project

The **AI-Powered Medical Assistant & OTC Recommendation System** is a comprehensive web application designed to bridge the gap between initial symptom onset and professional medical care. The project integrates supervised machine learning models, safe medication recommendation engines, and secure web-based communication systems into a single platform capable of supporting patients and administrators.

The system enables users to register securely, enter symptoms through an intuitive interface, and receive AI-generated disease predictions alongside safe Over-The-Counter (OTC) medicine suggestions. Machine learning algorithms analyze symptom vectors to identify possible minor illnesses and generate helpful medical insights, usage instructions, dosages, and potential side effects.

The platform also maintains a persistent history of previous symptom searches and predictions for each registered user, allowing them to review past results whenever needed. The administrative component enables authorized staff to execute full CRUD operations on disease data and medicine inventories, ensuring that all system information remains accurate and up-to-date. Overall, the project demonstrates how data-driven technologies can support healthcare awareness while strongly emphasizing responsible usage through mandatory medical disclaimers.

1.2 Problem Statement

Patients frequently rely on unverified internet search engines for self-diagnosis, which often results in severe misinformation, undue anxiety, or hazardous self-medication practices involving inappropriate prescription drugs or antibiotics. Existing web tools lack intelligent, safety-constrained triage mechanisms that guide users toward safe, non-prescription remedies while respecting strict clinical boundaries.

1.3 Objectives of the Proposed System

- To develop an accurate Random Forest machine learning prediction model for symptom-to-disease analysis.
- To implement a custom **Relative Confidence Margin** algorithm to mathematically evaluate and display model decisiveness transparently.
- To enforce a strict algorithmic **OTC safety filter** that recommends exclusively non-prescription medicines while excluding controlled substances.
- To provide robust user account management, history tracking, and downloadable PDF triage reports.
- To integrate **HTML5 Geolocation** for instant emergency routing to nearby hospitals.
- To establish a secure Admin Dashboard supporting dynamic Disease and Medicine CRUD operations.

1.4 Scope of the Proposed System

The scope of the project encompasses the design and development of an intelligent web platform centered around preliminary symptom triage and OTC medicine guidance. It supports

user registration, secure authentication, AI-driven prediction, history tracking, and administrative data management. While the current implementation focuses on common health conditions, minor ailments, and safe OTC recommendations, future expansions may include live hospital appointment booking APIs, deep learning natural language processing for free-text symptom descriptions, and native mobile application deployment.

CHAPTER 2

REQUIREMENTS SPECIFICATIONS

The requirement specification for the **AI-Powered Medical Assistant & OTC Recommendation System** outlines the development of an intelligent and secure web-based healthcare platform using modern technologies such as Python, the Flask framework, machine learning models, and integrated AI/ML libraries. The system is designed to assist users and administrators by providing AI-based symptom analysis, safe Over-The-Counter (OTC) medicine recommendations, prediction history tracking, and intelligent healthcare management services through a centralized digital environment. It incorporates advanced technologies including supervised machine learning algorithms for disease prediction, custom mathematical confidence margin algorithms, and strict programmatic safety filters.

Key functionalities of the system include secure user authentication and profile management, AI-driven symptom disease prediction modules, strict OTC medicine filtering, downloadable PDF report generation, and centralized administration dashboards. The platform enables users to enter symptoms through an intuitive interface to receive AI-generated predictions along with usage instructions, general dosages, possible side effects, and precautions. The machine learning models process symptom inputs and generate prediction confidence scores, which are normalized using a custom Relative Confidence Margin formula to provide transparent metrics.

The platform also supports efficient administrative interaction through a dedicated management module. Authorized administrators can manage user accounts, monitor system prediction logs, and execute full CRUD operations (Create, Read, Update, Delete) to maintain accurate and up-to-date disease and medicine databases. Real-time HTML5 Geolocation routing further connects users with nearby healthcare facilities during emergencies.

Overall, the project aims to deliver a scalable, secure, and user-friendly healthcare solution that bridges the gap between initial symptom onset and professional medical care, thereby improving preliminary health accessibility, supporting early disease awareness, enhancing operational efficiency, and promoting responsible self-care through appropriate medical disclaimers.

2.1 OVERALL DESCRIPTION

The project is a comprehensive web-based healthcare application developed using modern technologies such as Python and the Flask framework, integrated with advanced artificial intelligence and machine learning models. The platform is designed to provide an intelligent and interactive health environment that assists users and administrators in managing preliminary healthcare guidance efficiently. It offers a robust backend system that manages

disease prediction, user histories, OTC medicine databases, and administrative controls while ensuring secure and reliable data processing.

Addressing the growing need for safe and structured health information online, the system enables users to input symptoms, receive AI-generated disease predictions, and access safe non-prescription medicine recommendations in real time. By incorporating features such as machine learning-based risk assessment, strict OTC safety filtering, prediction history tracking, and an administrative CRUD dashboard, the platform ensures a comprehensive and user-friendly experience while emphasizing responsible usage through mandatory medical disclaimers.

2.1.1 Functional Requirements:

- **User Authentication & Management:** The platform enables users to register securely, log in using hashed credentials (Werkzeug PBKDF2), and recover forgotten passwords via email workflows.
- **AI-Driven Disease Prediction:** Users can input symptoms and health parameters, which are processed by a Random Forest Classifier to generate disease predictions and normalized confidence scores.
- **Strict OTC Safety Filtering:** The system enforces an algorithmic filter ensuring that only verified Over-The-Counter (OTC) medications are recommended while prescription drugs and narcotics are excluded.
- **History Tracking & Reporting:** Authenticated users can review past predictions and generate downloadable PDF medical reports using ReportLab utilities.
- **Geolocation Hospital Routing:** Integrates the HTML5 Geolocation API to instantly fetch user coordinates and route them to nearby hospitals via Google Maps.
- **Admin CRUD Management:** Authorized administrators can securely log in via isolated .env credentials to manage diseases, precautions, and medicine inventories through a fully functional dashboard.

2.2 PRODUCT PERSPECTIVE

The platform is positioned as an advanced web-based medical assistant designed to improve preliminary healthcare accessibility, disease prediction, and safety-constrained medicine guidance through the integration of artificial intelligence, machine learning, and modern web technologies. Developed using Python and the Flask framework (with modular Blueprint routing), it provides a scalable, secure, and efficient architecture.

At the core of the system, a MongoDB Atlas NoSQL cloud database efficiently manages user profiles, prediction histories, disease mappings, and medicine inventories. The structured BSON document design ensures secure data storage, fast retrieval, and efficient tracking of user health history over time. Flask provides a robust backend infrastructure managing routing, session handling, and communication between machine learning models and the Tailwind CSS frontend.

2.3 PRODUCT FUNCTIONS

- **Symptom Input & Vectorization:** Translates user-selected symptoms into binary feature arrays for machine learning ingestion.
- **AI Prediction & Confidence Scoring:** Executes the Random Forest model and computes the custom Relative Confidence Margin for transparency.
- **OTC Medication Matching:** Queries the database to retrieve safe, non-prescription treatments matching the diagnosed condition.
- **PDF Report Compilation:** Compiles triage details into a downloadable document format for personal records or physician consultation.
- **Emergency Geolocation Integration:** Connects browser coordinate detection with map search queries for urgent care routing.
- **Admin Dashboard Operations:** Provides centralized control for data management, user monitoring, and system metrics visualization.

2.4 USER CHARACTERISTICS

- **Standard User / Patient:** Individuals seeking preliminary health triage, common illness identification, and safe OTC medicine guidance.
- **Administrator:** Authorized personnel responsible for system maintenance, security oversight, and database CRUD operations for diseases and medicines.

2.5 OPERATING ENVIRONMENT CONSTRAINTS

2.5.1 Software Specifications

- **Operating System:** Windows 10/11, macOS, or Linux
- **Web Browser:** Any Standard Web Browser
- **Frontend:** HTML5, CSS3, Tailwind CSS, JavaScript
- **Backend:** Python Flask
- **Database:** MongoDB Atlas
- **Programming Language:** Python 3.10+
- **Machine Learning Libraries:** Scikit-learn, Pandas, NumPy, Joblib
- **Development Environment:** VS Code / PyCharm
- **Version Control:** Git & GitHub

2.5.2 Hardware Specifications

- **Processor:** Intel i5 / AMD Ryzen 5 or higher
- **RAM:** 8GB Minimum
- **Storage:** 1GB+ Free Disk Space
- **Display:** Standard Monitor with 1366×768 Resolution or Higher

- **Peripherals:** Keyboard and Mouse

2.6 SPECIFIC REQUIREMENTS

- **2.6.1 User Requirements:** Secure registration, password encryption (Werkzeug PBKDF2), password recovery, profile management, and symptom submission.
- **2.6.2 Admin Requirements:** Complete control and monitoring capabilities, database CRUD operations, user management, and system analytics.
- **2.6.3 Functional Requirements:** Secure authentication, AI-based disease prediction, strict OTC safety filtering, history tracking, and PDF report generation.
- **2.6.4 Non-Functional Requirements:** High system performance, reliability, scalability, responsiveness across devices, and low response times.
- **2.6.5 Security Requirements:** HTTPS protocol for secure transmission, password encryption, role-based access control (RBAC), and isolated admin credentials.

2.7 EXTERNAL INTERFACE REQUIREMENTS

The external interface requirements define how the platform interacts with users, browsers, databases, and external services like the HTML5 Geolocation API and SMTP email dispatch. The user interface provides a clean, responsive dark-mode SaaS design compatible across desktops, tablets, and smartphones.

2.8 SYSTEM FEATURES

- **AI-Based Disease Prediction:** Dynamic analysis of symptoms using Random Forest classification and custom Relative Confidence Margin scoring.
- **Strict OTC Safety Filtering:** Algorithmic exclusion of prescription drugs and narcotics to ensure safe self-care recommendations.
- **Interactive Triage Services:** User-friendly forms and dashboards for symptom entry, history tracking, and report downloads.
- **Admin CRUD Dashboard:** Centralized control center for managing diseases, precautions, and medicine inventories.
- **Geolocation Hospital Routing:** Instant mapping of nearby healthcare facilities using browser geolocation coordinates.

CHAPTER 3

SYSTEM DESIGN AND TEST PLAN

System design refers to the process of defining and developing the architecture, components, modules, interfaces, workflows, and data management structure of a system based on the requirements identified during the analysis phase. It serves as the foundation for transforming conceptual healthcare ideas into a fully functional intelligent software solution. In the **AI-Powered Medical Assistant & OTC Recommendation System**, system design focuses on creating a secure, scalable, intelligent, and efficient web platform capable of supporting AI-based symptom prediction, strict Over-The-Counter (OTC) medicine filtering, prediction history tracking, and administrative CRUD management through the integration of Python Flask and MongoDB Atlas.

The system design process is developed based on the functional and non-functional requirements identified during analysis. It involves planning the overall architecture of the web platform, defining the interaction between modules (via Flask Blueprints), and establishing the flow of data throughout the system. The design ensures that the proposed system fulfills its primary objectives of providing AI-based symptom analysis, safe non-prescription medicine recommendations, PDF report generation, and secure administrative controls.

The platform is designed as a web-based healthcare application using Python and the **Flask framework** integrated with machine learning libraries (Scikit-learn, Pandas, NumPy, Joblib) and cloud database connectivity (PyMongo). The system architecture consists of multiple interconnected modules including user authentication (Werkzeug PBKDF2), symptom vectorization, Random Forest inference engine, Relative Confidence Margin calculation, strict OTC safety filtering, prediction history logging, and admin CRUD management. Each module performs specific operations while maintaining smooth data exchange.

System design also includes the development of a structured NoSQL database architecture for storing user credentials, prediction histories, disease records, and medicine inventories securely in MongoDB Atlas collections. Proper database design ensures efficient data retrieval, secure storage, and scalability. The backend logic manages communication between the Tailwind CSS frontend interface, the machine learning models, and the cloud database.

Additionally, the system design incorporates advanced security mechanisms such as Werkzeug password hashing, environment variable (.env) isolation for administrative credentials, role-based access control (RBAC), and protected data storage to maintain user privacy and data integrity. Real-time processing capabilities support symptom analysis, geolocation routing, and report generation with minimal latency.

3.1 DATA FLOW DIAGRAM (DFD)

A data flow diagram (DFD) is a graphical representation of the flow of data within a system. It depicts how data is input, processed, stored, and output in a system or process. DFDs are widely used in software development, system analysis, and business process modelling to understand and communicate the data flow and interactions between different components of a system.

A level 0 DFD, also called a fundamental system model or a context model, represents the entire software elements as a single bubble with input and output indicated by incoming and outgoing arrows respectively. Additional process and information flow parts are represented in the next level i.e., Level 1 DFD. Each of the processes represented at Level 1 are sub-functions of the overall system depicted in the context model.

Data flow diagrams illustrate how data is processed by a system in terms of inputs and outputs. Major components or functions are represented with Circles. Actions for input by a user or a system go in Rectangular Boxes. Databases are represented by Parallel lines enclosing a phrase corner.

3.1.1 Level 0 DFD

The Level 0 DFD of the AI Healthcare Management System provides a high-level representation of how data flows between the major entities and processes within the system. It illustrates the interaction between the two primary external entities, namely the Patient and the Admin, with the central healthcare management system. The Level 0 DFD focuses on the overall functionality of the platform without going into the internal sub-processes, thereby presenting a clear overview of system operations and information exchange.

In the proposed system, the Patient acts as one of the primary external entities who interact directly with the AI Healthcare Management System. The patient begins by registering and logging into the platform using valid credentials. The system processes the authentication details and grants secure access to the patient dashboard. Once authenticated, the patient can submit symptoms and clinical data to the AI Diagnostic Centre, which processes the input through pre-trained machine learning models to generate disease risk predictions. These predictions include risk scores, medical insights, and relevant specialist recommendations, which are returned to the patient through the system dashboard.

The system also enables the patient to book appointments with verified doctors by selecting preferred time slots and consultation modes. The appointment details are transferred to the Appointment Management Process, which coordinates scheduling between the patient and the practitioner. The processed data, including prediction history, appointment records, and user profile details, are stored in the central database for future reference and doctor review.

The Admin acts as the second major external entity in the Level 0 DFD. The admin interacts with the system through the Admin Control Panel. The admin can verify and approve new doctor and hospital registrations, monitor user activities, manage user accounts, and analyse overall system performance. Administrative data and analytics are retrieved from the central database and displayed through the admin dashboard. The admin also ensures proper system maintenance, verifies data integrity, and oversees the functioning of AI diagnostic processes.

The central database plays a critical role in the Level 0 DFD by storing and managing all essential system information, including user profiles, doctor details, hospital records,

appointment sessions, AI prediction results, feedback reports, and admin-related data. All major processes interact with the database to retrieve and update information efficiently.

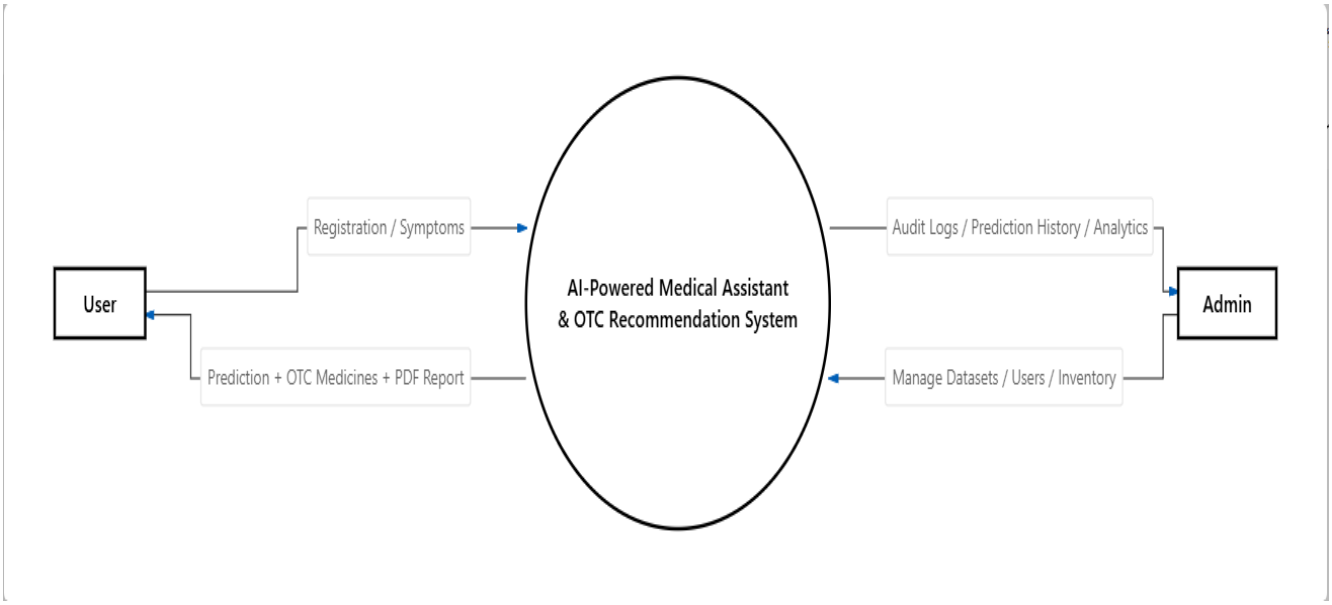


Figure 3.1: DFD Level 0

3.1.2 Level 1 DFD

The Level 1 DFD illustrates the detailed workflow and interaction between different modules, users, processes, and databases within the system. The diagram represents how data flows from external entities such as users and administrators to various functional modules of the application. The process begins with the User Registration and Authentication module, where users enter their credentials, which are securely verified and hashed using Werkzeug PBKDF2 to grant system access. After successful login, users can perform different operations based on their roles.

The AI Disease Prediction module allows users to input symptoms and health parameters into the system. The entered data is processed by the Random Forest prediction engine, which computes disease predictions and normalizes confidence scores using the Relative Confidence Margin algorithm. These prediction results, along with filtered Over-The-Counter (OTC) medicine recommendations, are stored in the prediction_history collection in MongoDB Atlas for future reference and analysis. The OTC Safety Filtering module ensures that only verified non-prescription medicines are retrieved from the medicines collection while excluding prescription drugs and narcotics.

The Admin CRUD Management and Monitoring modules handle administrative activities. Authorized administrators can securely log in via isolated environment credentials to execute full CRUD operations (Create, Read, Update, Delete) on diseases, precautions, and medicine inventories stored in the cloud database. The admin module also helps administrators analyze system activities, monitor prediction logs, and manage user accounts.

Additionally, the HTML5 Geolocation module allows users to fetch browser coordinates and instantly route to nearby hospitals during emergencies. Overall, the Level 1 DFD clearly describes the internal processes, NoSQL data collections, and flow of information within the proposed AI-Powered Medical Assistant and OTC Recommendation System.

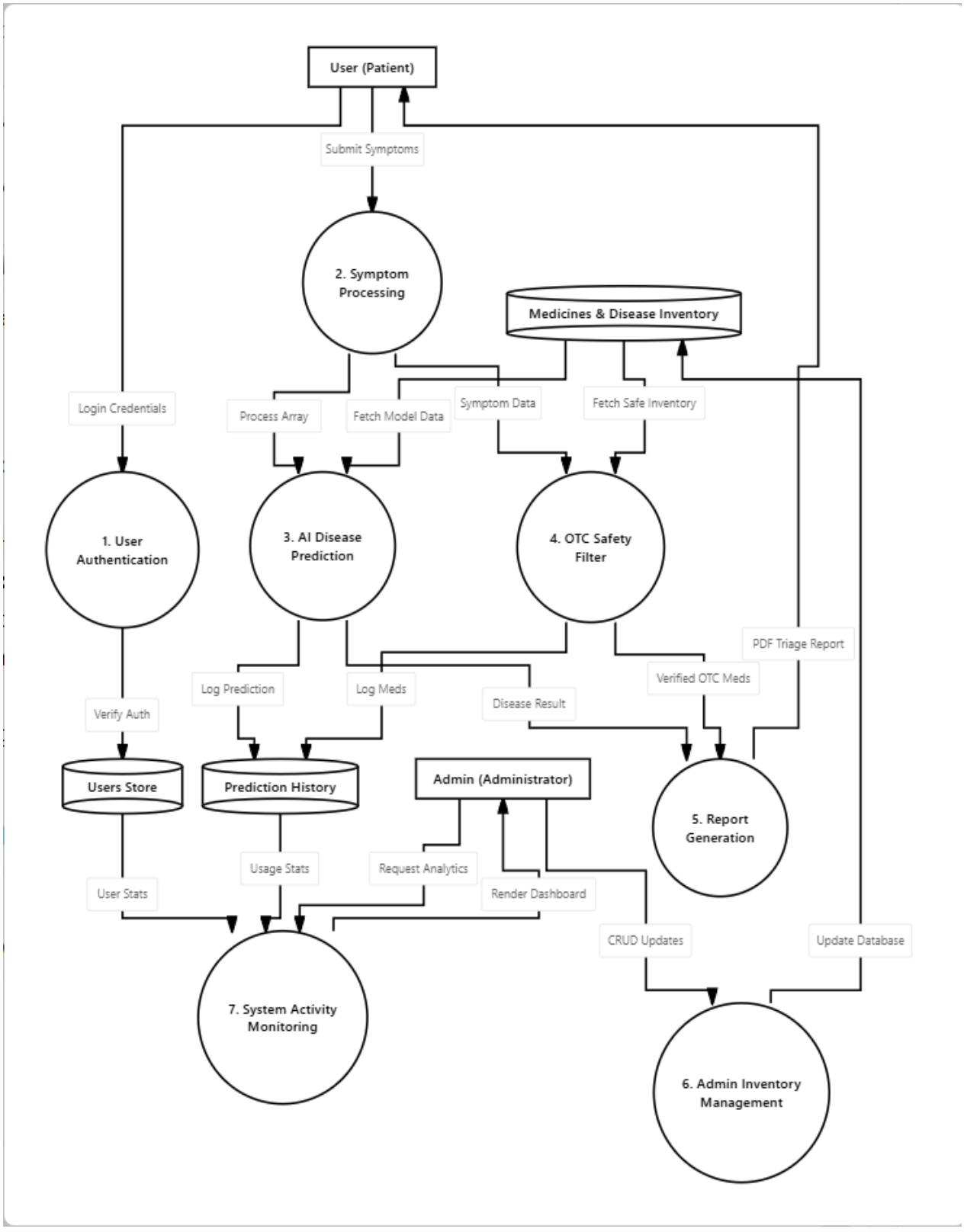


Figure 3.2: DFD Level 1

3.2 USE CASE DIAGRAM

The Use Case Diagram of the **AI-Powered Medical Assistant & OTC Recommendation System** illustrates the interaction between the primary actors of the system and the major functionalities provided by the platform. The diagram consists of two actors: the Admin and the User (Patient). It represents how each actor interacts with different modules of the system to perform specific operations related to user authentication, AI-driven symptom analysis, safe Over-The-Counter (OTC) medication recommendations, PDF report generation, and administrative database management. The use case diagram provides a high-level functional overview of the system and helps in understanding the responsibilities and interactions of each user role within the platform.

3.2.1 Admin Use Case Diagram

The Admin actor is responsible for managing and monitoring the overall activities of the system through an isolated, secure control panel. One of the primary use cases associated with the admin is **Manage Diseases & Precautions**, where the admin executes full CRUD operations (Create, Read, Update, Delete) on disease records and safety guidelines stored in MongoDB Atlas. Another important functionality is **Manage Medicines**, which allows the admin to add, update, or remove Over-The-Counter (OTC) medicines while verifying safety flags such as is_otc: True.

The admin also interacts with the **Manage Users** use case, enabling oversight of registered accounts, monitoring user engagement, and ensuring the overall security of the platform. Through the **Monitor Prediction Logs** use case, the admin can review all AI-generated diagnostic records and prediction histories across the user base to ensure system reliability. Additionally, the **System Maintenance & Analytics** use case allows the admin to oversee platform operations, verify environment security, and maintain the integrity of the machine learning prediction datasets.

3.2.2 User Use Case Diagram

The User (Patient) actor represents the primary individual interacting with the web platform for preliminary health triage and safe self-care guidance. The primary use case for the user is **Secure Authentication**, where individuals register and log in securely using credentials hashed with Werkzeug PBKDF2. Once authenticated, the user engages with the **Input Symptoms & Parameters** use case to submit health symptoms into the system.

The user then interacts with the **View AI Disease Prediction** use case, where the backend executes the Random Forest classification model and displays predicted diseases alongside the custom Relative Confidence Margin score. Following this, the **View OTC Recommendations** use case provides safe, non-prescription medicine suggestions while strictly filtering out prescription drugs and narcotics. Additional user use cases include **Download PDF Report** (compiling triage summaries via ReportLab), **Track Prediction History** (viewing past health searches stored in MongoDB Atlas), and **Find Nearby Hospitals** (triggering the HTML5 Geolocation API for emergency map routing).

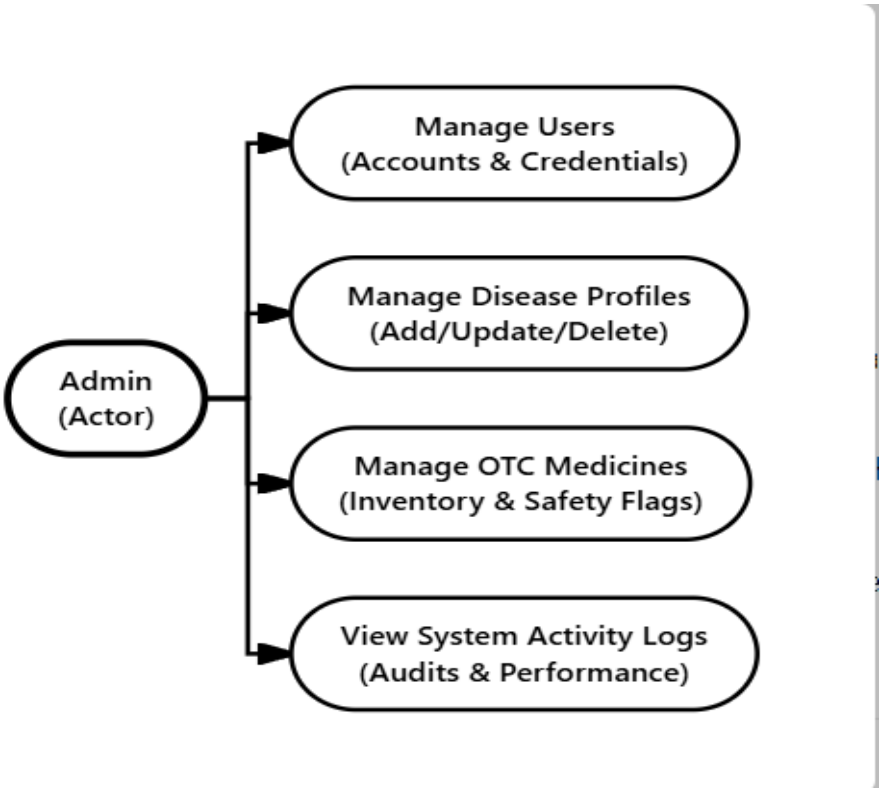


Figure 3.3: Admin Use Case Diagram

.2.2 User (Patient) Use Case Diagram

The User (Patient) actor represents the primary individual who utilizes the web platform for preliminary health triage and safe self-care guidance. The first interaction of the user with the system occurs through the **Register/Login** use case, where users create accounts and securely access the platform using valid credentials hashed with Werkzeug PBKDF2. After authentication, the user proceeds to the **Input Symptoms & Parameters** use case, where health symptoms are submitted through intuitive web forms for AI-based analysis.

Following the input, the user interacts with the **View AI Disease Prediction & OTC Recommendations** use case to receive detailed results, including Random Forest classification predictions, the custom Relative Confidence Margin score, and verified Over-The-Counter (OTC) medicine suggestions while strictly filtering out prescription drugs and narcotics. The **Download PDF Report** use case allows the user to compile and download structured medical triage reports generated via ReportLab utilities.

The **Track Prediction History** use case provides the user with a chronological record of all past AI diagnostic assessments stored in MongoDB Atlas, enabling them to monitor their health progress over time. Finally, the **Find Nearby Hospitals** use case enables the user to trigger the HTML5 Geolocation API to fetch browser coordinates and instantly route to nearby healthcare facilities via Google Maps during emergencies.

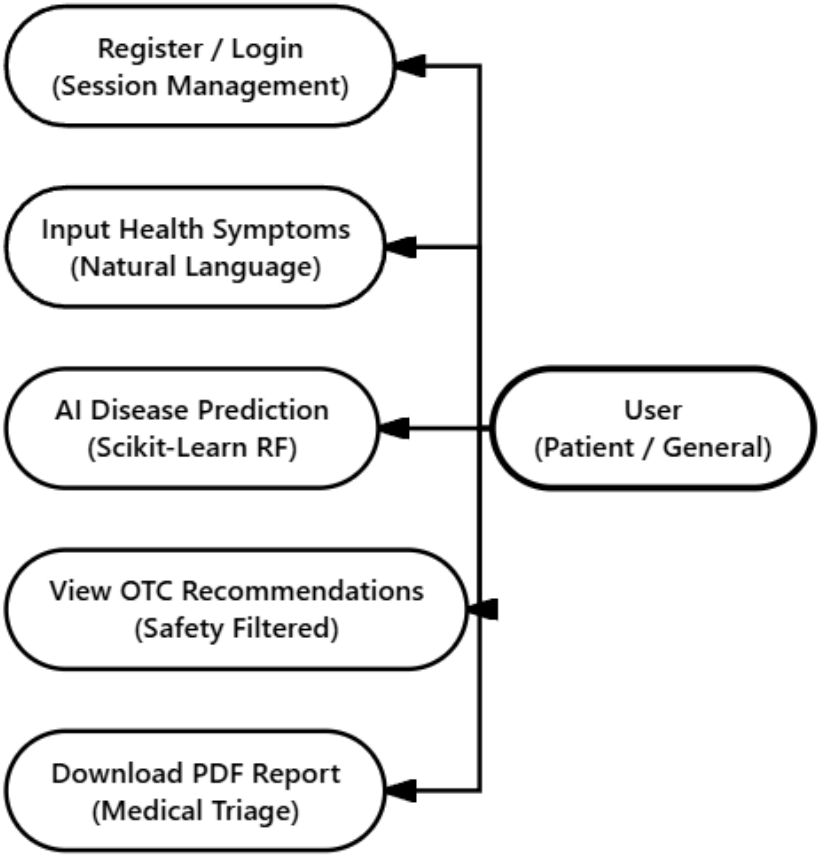


Figure 3.4: User Use Case Diagram

3.2.3 Administrative & System Management Use Case Diagram

The Administrative Management Module in the use case diagram allows administrators to manage overall system operations, database collections, and user activities securely. After successful environment-authenticated login, administrators can execute full CRUD operations (Create, Read, Update, Delete) on disease records, precautions, and Over-The-Counter (OTC) medicine inventories. The module enables administrators to monitor system-wide AI prediction histories, review prediction confidence metrics, and manage user accounts by verifying or restricting access. Additionally, administrators can oversee database integrity across MongoDB Atlas collections and ensure that all medicine recommendations strictly comply with non-prescription safety filters.

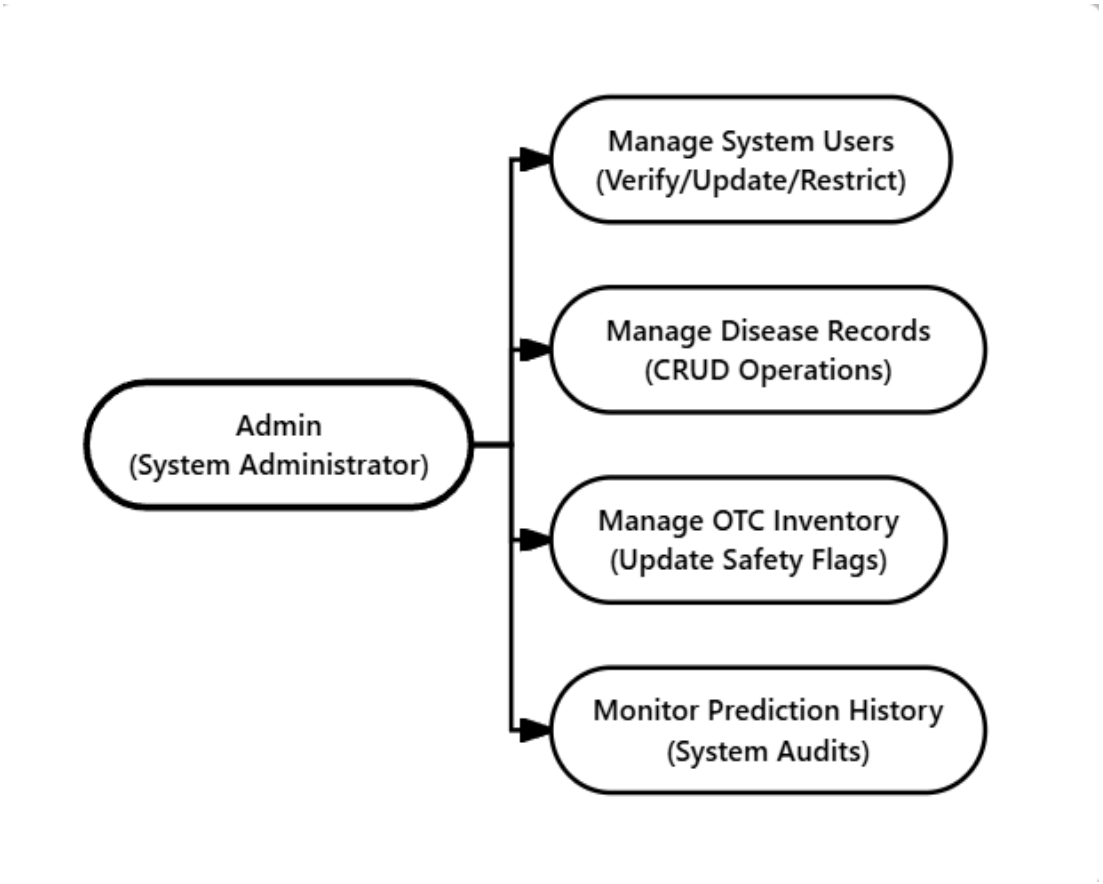


Figure 3.5: Administrative & System Management Use Case Diagram

3.3 DATABASE DESIGN (MONGODB ATLAS COLLECTIONS)

The system utilizes MongoDB Atlas, a fully managed NoSQL cloud database, to store and manage application data efficiently using flexible BSON document structures. Unlike traditional relational databases, MongoDB collections provide high scalability, rapid retrieval, and schema flexibility required for handling user profiles, prediction histories, and medical inventories.

3.3.1 Users Collection (users)

- Stores core user identity, authentication credentials, and profile attributes for registered individuals within the platform.

Table 3.1: users Collection Schema

Field	Data Type	Constraints	Description
_id	ObjectId	Primary Key	Unique MongoDB identifier for each user document
full_name	String	Not Null	Full name of the registered user
email	String	Unique, Not Null	User's email address used for login and identification
password	String	Not Null	Securely hashed password using Werkzeug PBKDF2
age	Integer	Nullable	Optional age parameter of the user
gender	String	Nullable	Optional gender parameter of the user
medical_history	String	Nullable	Optional background medical history notes
created_at	DateTime	Default Current Timestamp	Stores account creation date and time

3.3.2 Prediction History Collection (prediction_history)

- Stores detailed health triage logs, AI-generated disease predictions, confidence metrics, and safe Over-The-Counter (OTC) medicine recommendations for registered users.

Table 3.2: prediction_history Collection Schema

Field	Data Type	Constraints	Description
_id	ObjectId	Primary Key	Unique MongoDB identifier for each prediction log
user_id	ObjectId	Reference / Not Null	References the registered user ID from the users collection
symptoms	Array of Strings	Not Null	List of symptoms inputted by the user
predicted_disease	String	Not Null	Disease predicted by the Random Forest classifier
confidence_score	Float / String	Not Null	Normalized confidence percentage based on the Relative Confidence Margin
recommended_medicines	Array of Strings	Not Null	Filtered list of safe Over-The-Counter (OTC) medicines
timestamp	DateTime	Default Current Timestamp	Stores the exact date and time of the AI diagnostic session

3.3.3 Medicines Collection (medicines)

- Stores Over-The-Counter (OTC) medicine details, usage instructions, dosages, side effects, and clinical safety flags managed through the administrative CRUD dashboard.

Table 3.3: medicines Collection Schema

Field	Data Type	Constraints	Description
_id	ObjectId	Primary Key	Unique MongoDB identifier for each medicine document
name	String	Unique, Not Null	Brand or generic name of the medicine
category	String	Not Null	Disease focus or medical category associated with the medicine
description	String	Not Null	Detailed usage instructions and guidelines
dosage	String	Not Null	General recommended dosage information
side_effects	String	Nullable	Potential side effects or warnings
is_otc	Boolean	Not Null (Default = True)	Crucial safety flag ensuring only non-prescription medicines are recommended
created_at	DateTime	Default Current Timestamp	Stores medicine entry creation date and time

3.3.4 Diseases Collection (diseases)

- Stores disease profiles, medical descriptions, and associated precautionary guidelines managed through the administrative database.

Table 3.4: diseases Collection Schema

Field	Data Type	Constraints	Description
_id	ObjectId	Primary Key	Unique MongoDB identifier for each disease document
name	String	Unique, Not Null	Name of the disease or medical condition
description	String	Not Null	Detailed clinical description of the condition
precautions	Array of Strings	Not Null	Recommended safety measures and precautions
created_at	DateTime	Default Current Timestamp	Stores disease entry creation date and time

3.3.5 Disease Prediction Collection (prediction_history)

- Stores AI-generated disease prediction results, symptom logs, confidence metrics, and safe Over-The-Counter (OTC) medication recommendations for users.

Table 3.5: prediction_history Collection Schema

Field	Data Type	Constraints	Description
_id	ObjectId	Primary Key	Unique MongoDB identifier for each prediction record
user_id	ObjectId	Reference / Not Null	References the registered user in the users collection
symptoms	Array of Strings / Text	Not Null	Symptoms entered by the user for analysis
predicted_disease	String	Not Null	Disease predicted by the Random Forest classifier

Field	Data Type	Constraints	Description
confidence_score	Float / String	Not Null	Confidence percentage or Relative Confidence Margin score
recommendations	Array of Strings / Text	Nullable	Suggested safe Over-The-Counter (OTC) medical recommendations
created_at	DateTime	Default Current Timestamp	Stores prediction date and time

3.3.6 Diseases Collection (diseases)

- Stores disease profiles, clinical descriptions, and associated precautionary guidelines managed through the administrative database.

Table 3.6: diseases Collection Schema

Field	Data Type	Constraints	Description
_id	ObjectId	Primary Key	Unique MongoDB identifier for each disease document
name	String	Unique, Not Null	Name of the disease or medical condition
description	String	Not Null	Detailed clinical description of the condition
precautions	Array of Strings / Text	Not Null	Recommended safety measures and precautions
created_at	DateTime	Default Current Timestamp	Stores disease entry creation date and time

3.3.7 Healthcare Report Collection (prediction_history / Generated Reports)

- Stores structured medical triage summaries, confidence scores, and safety recommendations compiled for PDF report generation.

Table 3.7: healthcare_report Collection Schema

Field	Data Type	Constraints	Description
_id	ObjectId	Primary Key	Unique MongoDB identifier for each report record
user_id	ObjectId	Reference / Not Null	References the registered user in the users collection
prediction_id	ObjectId	Reference / Not Null	References the specific prediction entry from the prediction logs
report_summary	Text / String	Not Null	Summary of the AI-generated health prediction report
risk_level	String	Not Null	Indicates triage severity or confidence category
recommendations	Array of Strings / Text	Nullable	Suggested safe Over-The-Counter (OTC) guidance and precautions
generated_at	DateTime	Default Current Timestamp	Stores report generation date and time

3.3.8 System Activity Logs Collection (activity_logs)

- Stores administrative audit trails, user action tracking, and system-wide diagnostic monitoring data within the MongoDB Atlas database.

Table 3.8: activity_logs Collection Schema

Field	Data Type	Constraints	Description
_id	ObjectId	Primary Key	Unique MongoDB identifier for each activity log document
user_id	ObjectId / String	Reference / Not Null	References the user or administrator who performed the action
action_type	String	Not Null	Type of action performed (e.g., User Login, AI Prediction, OTC Inventory CRUD)
description	Text	Not Null	Detailed description of the system event or database modification
ip_address	String	Nullable	IP address from where the web request originated
created_at	DateTime	Default Current Timestamp	Stores the exact date and time of the logged system activity

3.4 PERFORMANCE REQUIREMENTS

Performance requirements are essential to ensure the efficient operation, responsiveness, reliability, and scalability of the **AI-Powered Medical Assistant & OTC Recommendation System**. The platform should deliver fast response times for critical operations such as secure user authentication (Werkzeug PBKDF2), Random Forest disease prediction processing, safe Over-The-Counter (OTC) medicine filtering, PDF report generation, and administrative CRUD operations. Real-time processing capabilities are highly important, especially for machine learning inference and database retrieval, ensuring that users experience smooth and uninterrupted interactions without noticeable delays during system usage.

The platform must be scalable to support an increasing number of registered users, prediction requests, and concurrent database activities without performance degradation. The system should efficiently handle multiple users simultaneously, allowing individuals to access symptom prediction modules, track prediction histories, and administrators to manage medical inventories in real time. High availability and reliability are critical to ensure continuous service delivery with minimal downtime, enabling users to access health triage guidance whenever required.

Efficient data processing and secure data management are also important performance considerations. The system must securely process and store sensitive user credentials, prediction logs, and medical inventories while maintaining quick data retrieval and response times. The cloud backend database (MongoDB Atlas) should be optimized for efficient document storage, indexing, and retrieval to ensure smooth system operation even with expanding datasets and increasing user activity.

The machine learning prediction modules and safety filtering algorithms must execute efficiently to generate accurate disease predictions, relative confidence margin scores, and verified non-prescription medicine suggestions within minimal processing time. The platform should support responsive UI rendering for data visualization, report generation, and administrative dashboards to improve user engagement. Efficient communication between the Flask web framework, Scikit-learn models, and MongoDB collections is necessary to maintain seamless system operations.

Proper error handling, secure environment variable configurations, and recovery mechanisms must be implemented to reduce service interruptions and maintain operational continuity during unexpected failures, system crashes, or network issues. Additionally, the web application interface must remain responsive across desktops, tablets, and smartphones, ensuring a consistent and accessible healthcare guidance experience under different device and internet conditions.

3.5 SOFTWARE QUALITY ATTRIBUTES

For the **AI-Powered Medical Assistant & OTC Recommendation System**, software quality attributes are essential to ensure that the platform delivers reliable, intelligent, secure, and efficient health triage services. Reliability is one of the most important quality attributes, ensuring that the system consistently performs operations such as Random Forest disease prediction, safe Over-The-Counter (OTC) medicine filtering, PDF report generation, and administrative management without failures or interruptions. The platform must maintain stable operations during continuous usage and concurrent activities.

Usability focuses on providing an intuitive and user-friendly interface that allows users and administrators to interact with the system easily. The platform should provide simple navigation, organized SaaS dashboards, clear health triage workflows, and accessible PDF reports, enabling users to operate the system without technical complexity. A smooth and interactive user experience is essential for improving health accessibility, patient engagement, and administrative management efficiency.

Security is a major quality attribute of the platform because it manages sensitive user accounts, prediction histories, and medical inventories. The system must implement secure authentication mechanisms (Werkzeug PBKDF2 password hashing), isolated environment variable controls, role-based access control, and secure cloud database storage practices (MongoDB Atlas) to prevent unauthorized access and maintain data privacy. Continuous validation, secure authorization mechanisms, and strict input filtering are necessary to strengthen protection against potential security threats.

Performance is another critical quality attribute, requiring the system to deliver fast response times for disease prediction processing, safety filtering, PDF report generation, and database retrieval. The platform must efficiently manage multiple concurrent users and real-time machine learning inference without performance degradation. Low latency in prediction analysis and report compilation is essential for maintaining an effective and responsive environment.

Scalability ensures that the platform can accommodate increasing numbers of registered users, prediction records, and database documents without affecting efficiency or responsiveness. The system architecture should support future expansion and integration with advanced technologies such as expanded disease datasets, cloud-based analytics, and enhanced machine learning models.

Maintainability is essential for enabling easy system updates, bug fixes, machine learning model improvements, and feature enhancements. The platform is designed using a modular and well-structured Flask architecture with properly organized code, making it easier to maintain, upgrade, and extend over time. This improves long-term adaptability, software management efficiency, and system sustainability.

Compatibility ensures that the web application functions efficiently across different devices, browsers, and operating systems. The responsive HTML5/CSS interface allows users and administrators to access services from desktops, laptops, tablets, and smartphones while maintaining a consistent and reliable user experience.

High availability is another essential quality attribute to ensure uninterrupted access and reliable operations. The system should minimize downtime through proper error handling, fault tolerance strategies, and optimized server performance. Continuous availability improves user trust and ensures uninterrupted service delivery and management operations.

Finally, testability is an important software quality attribute that ensures the platform can be thoroughly validated through unit testing, integration testing, system testing, and functional testing. Comprehensive testing guarantees that all modules, including the Random Forest prediction engine, OTC safety filtering system, PDF reporting utilities, and administrative CRUD dashboards, operate accurately, securely, and reliably under different usage conditions.

3.6 DETAILED DESIGN

The detailed design of the **AI-Powered Medical Assistant & OTC Recommendation System** involves a comprehensive approach toward system architecture, web workflows, machine learning integration, NoSQL database management, user interaction, security implementation, and administrative controls to satisfy both functional and non-functional requirements effectively.

3.6.1 System Architecture:

The platform is developed using Python and the Flask framework with a modular and scalable architecture to ensure maintainability, reliability, flexibility, and efficient health triage operations. The frontend provides a responsive and interactive web interface built with HTML5, CSS, and JavaScript that allows users and administrators to access the platform

efficiently through desktops, tablets, and smartphones. The interface enables users to register securely, submit health symptoms to machine learning prediction modules, view verified Over-The-Counter (OTC) medicine recommendations, download structured PDF reports, track prediction histories, and locate nearby hospitals using the HTML5 Geolocation API.

The backend, powered by Flask, manages the core application business logic and coordinates communication between the frontend interface, cloud database, machine learning models, and security utilities. The platform integrates Scikit-learn machine learning technologies, Random Forest classification algorithms, and a custom Relative Confidence Margin decision system. MongoDB Atlas is used as the primary cloud database for storing user profiles, disease descriptions, medicine inventories, and prediction history logs. The modular system architecture ensures smooth interaction between authentication modules, AI prediction engines, safety filtering systems, and reporting components while supporting future scalability and expansion of medical datasets.

3.6.2 User Registration and Authentication:

Users can register on the platform using a valid email address, full name, and secure password. Passwords are securely encrypted using Werkzeug PBKDF2 hashing algorithms before storage in MongoDB Atlas to ensure data privacy and account security. The authentication process is managed using Flask session handling, which manages login states, user verification, and access permissions securely.

Role-based access control ensures that standard registered users (patients/individuals) and administrators can only access functionalities authorized for their respective roles. Standard users can submit health symptoms, view AI predictions, and download PDF reports, while administrators have exclusive access to the CRUD control panel for managing disease profiles, OTC medicine inventories, and system activity logs. This improves overall system security and prevents unauthorized access to sensitive application data.

3.6.3 User Profile and Healthcare Management:

Users and administrators can manage and update their accounts and system data through dedicated interfaces. The system stores and maintains user credentials, AI prediction histories, symptom logs, and administrative inventory databases securely within MongoDB Atlas. Personalized dashboards allow users to access their past prediction reports, monitor historical health triage trends, and view verified Over-The-Counter (OTC) medicine recommendations.

Administrators have access to a centralized management dashboard to execute full CRUD operations on disease records and OTC medicine inventories, monitor system-wide AI prediction metrics, and review activity audit logs. The management module ensures smooth coordination, data integrity, and efficient workflow management throughout the platform.

3.6.4 AI-Based Disease Prediction and OTC Recommendation:

The system utilizes machine learning models and predictive analytics algorithms (specifically a Random Forest classifier) to dynamically analyze user-submitted health symptoms and parameters. Users can input multiple health symptoms through structured web forms before

initiating diagnostic prediction requests. The prediction engine processes these inputs and generates disease risk predictions, confidence scores, and verified Over-The-Counter (OTC) medicine recommendations.

The platform controls workflows by processing symptom analysis requests, filtering out restricted prescription drugs to ensure non-prescription safety, and maintaining records. The system also tracks user prediction histories and past diagnostic results stored in MongoDB Atlas to support long-term health tracking and continuity.

3.6.5 Symptom Data Processing and Model Evaluation:

Users can submit health information through structured web forms and symptom selection interfaces. The prediction engine processes submitted data using trained Scikit-learn machine learning algorithms and intelligent classification techniques. Inputs are analyzed to determine probable disease conditions, symptom severity indicators, prediction confidence scores, and safe medication suggestions.

The evaluation engine compares user symptom parameters with trained machine learning feature models to generate accurate health assessments. The system evaluates prediction metrics and confidence margins to ensure that only relevant disease matches and verified safe OTC treatments are recommended to the user.

3.6.6 PDF Reporting and Administrative Analytics:

The system generates detailed downloadable medical triage reports after each disease prediction process using Report Lab utilities. Reports include predicted disease results, confidence scores, symptom summaries, and safe Over-The-Counter (OTC) medicine recommendations. Structured PDF generation enables users to easily save, print, or review their health triage assessments.

Administrators can access secure analytical dashboards to monitor system-wide prediction activities, review prediction trends, manage medicine and disease inventories, and evaluate overall platform performance. This functionality supports efficient system administration, operational monitoring, and continuous platform improvement.

3.6.7 Security Measures:

To ensure data security and user privacy, sensitive information such as user credentials, prediction histories, and medical inventories are securely managed and protected during storage and transmission. Passwords are securely hashed using Werkzeug PBKDF2 algorithms before storage in MongoDB Atlas. HTTPS protocols and environment variable configurations (.env) are implemented to secure communication and protect sensitive keys. Role-based access control restricts unauthorized access to administrative CRUD operations and protected application routes.

The platform incorporates secure authentication mechanisms, Flask session management, protected endpoint access, input data validation techniques, and structured error handling to identify and address potential security vulnerabilities. Secure database access practices, isolated administrative workflows, and protected communication channels further enhance system reliability and data protection.

3.6.8 Performance Optimization:

The platform is optimized to deliver fast response times and smooth real-time processing during disease prediction inference, safety filtering, PDF report generation, and database operations. Efficient NoSQL database queries and indexing mechanisms improve document retrieval speed for user profiles, prediction logs, and medicine inventories in MongoDB Atlas.

Optimized machine learning processing ensures quick generation of disease predictions, confidence margin scores, and verified Over-The-Counter (OTC) medicine recommendations. The modular system architecture supports scalability, allowing the platform to manage increasing numbers of users, prediction records, and concurrent web requests efficiently without significant performance degradation.

Optimized data processing techniques help maintain low latency during prediction analysis, ReportLab PDF compilation, and server communication processes, ensuring smooth operations and an improved user experience.

3.6.9 Testing and Deployment:

Comprehensive testing methodologies are followed to ensure platform quality, reliability, security, and functionality. Unit testing, integration testing, system testing, performance testing, security testing, and functional testing are performed to validate all application modules and ensure accurate system behavior. Special attention is given to testing machine learning prediction accuracy, Scikit-learn model evaluation metrics, OTC safety filtering reliability, and user authentication security functionalities.

The web platform can be deployed on cloud-based or local server environments using Python WSGI servers and modern hosting infrastructure for improved scalability, portability, and deployment efficiency. Structured deployment practices support efficient system updates, machine learning model improvements, maintenance operations, and future feature enhancements while ensuring minimal downtime and reliable service availability.

3.7.1 Test Case for Admin

Category	Test Case	Description	Expected Result
User Management	View All Users	Ensure the admin can view a list of all registered users on the platform.	A complete list of registered users is displayed with details such as full name, email, role, and registration date.
User Management	Manage User Status	Ensure the admin can oversee and	User account details and access statuses are managed successfully.

Category	Test Case	Description	Expected Result
		control registered user accounts.	
Medicine Inventory (CRUD)	Add / Edit OTC Medicines	Ensure the admin can add or update Over-The-Counter (OTC) medicines with safety flags.	New or modified medicine records, dosages, and is_otc safety flags are saved successfully in MongoDB Atlas.
Medicine Inventory (CRUD)	Delete OTC Medicine	Ensure the admin can remove invalid or outdated medicines from the inventory.	The selected medicine record is deleted successfully from the database.
Disease Management (CRUD)	Add / Edit Disease Profiles	Ensure the admin can add or update disease descriptions and clinical precautions.	Disease profiles and precautionary guidelines are updated correctly in the system.
Disease Management (CRUD)	Delete Disease Record	Ensure the admin can remove obsolete disease records from the database.	The selected disease entry is removed successfully.
Healthcare Monitoring	Monitor AI Prediction Logs	Ensure the admin can monitor system-wide AI-generated disease prediction history.	Prediction records including user identifiers, symptom inputs, predicted diseases, and confidence scores are displayed accurately.
System Audit	View Activity Logs	Ensure the admin can review system activity logs and audit trails.	Administrative and user system actions are displayed

Category	Test Case	Description	Expected Result
			chronologically with timestamps and details.
Category	Test Case	Description	Expected Result
Reporting & Analytics	Generate PDF Reports	Ensure the system generates downloadable health triage reports via ReportLab.	Structured PDF reports containing prediction results and safe OTC medicines are compiled and downloaded successfully.
Reporting & Analytics	View Dashboard Analytics	Ensure the admin dashboard displays system metrics and statistics correctly.	Dashboard displays total users, total predictions, and system statistics accurately.
Activity Monitoring	Monitor System Activities	Ensure the admin can track system operations and user activities.	Activity logs with timestamps, user actions, and IP details are displayed correctly.
Security & Access	Secure Admin Login	Ensure only authorized administrators can access the admin control panel.	Admin login succeeds with valid environment-authenticated credentials and blocks unauthorized access attempts.
Security & Access	Logout Functionality	Ensure the user or admin can securely log out from the platform.	The session is terminated successfully and the user is redirected to the login page.
System Maintenance	Error Handling Verification	Ensure the platform handles invalid inputs or system errors gracefully.	Appropriate error messages are displayed and system stability is maintained without crashing.

3.7.2 Test Case for User

Category	Test Case	Description	Expected Result
User Authentication	User Registration	Ensure the user can register using valid credentials with Werkzeug password hashing.	User account is created successfully and stored securely in MongoDB Atlas.
User Authentication	User Login	Ensure the user can log in using a valid email and password.	User is authenticated successfully via Flask sessions and redirected to the main dashboard.
User Authentication	Invalid Login	Ensure the system prevents login with incorrect credentials.	Error message is displayed for incorrect email or password.
Profile Management	View / Update Profile	Ensure the user can view and update personal account details.	User profile information is updated and saved successfully.
Disease Prediction	Input Symptoms	Ensure the user can select and submit health symptoms through structured web forms.	Symptom inputs are captured and transmitted successfully to the machine learning model.
Disease Prediction	View AI Prediction & OTC	Ensure AI-generated disease predictions and safe Over-The-Counter (OTC) medicines are displayed.	Predicted disease, confidence margin score, and filtered safe OTC medicines are displayed accurately.
Geolocation Routing	Find Nearby Hospitals	Ensure the user can fetch coordinates and route to nearby medical facilities.	HTML5 Geolocation API triggers successfully and opens

Category	Test Case	Description	Expected Result
			map routing to nearby hospitals.
Healthcare Reports	Download PDF Report	Ensure the user can download structured medical triage reports.	ReportLab compiles and downloads the medical assessment report successfully in PDF format.
Session History	View Prediction History	Ensure the user can access past health assessment records.	Chronological history of past AI predictions and recommendations from MongoDB is displayed correctly.
Security & Access	Secure User Logout	Ensure the user can securely log out from the platform.	The session is terminated successfully and the user is redirected to the login page.
Performance Testing	Concurrent User Handling	Ensure the Flask application handles multiple user requests simultaneously.	System performs efficiently without crashes or significant delays.
Error Handling	Invalid Input Validation	Ensure the system handles missing symptom inputs or invalid form data properly.	Appropriate validation warning messages are displayed to the user.
Device Compatibility	Responsive Interface Testing	Ensure the web application works properly across desktops, tablets, and smartphones.	User interface adapts correctly to different screen sizes and devices.

CHAPTER 4:

IMPLEMENTATION AND RESULTS

4.1 IMPLEMENTATION

The implementation of the **AI-Powered Medical Assistant & OTC Recommendation System** began with a detailed analysis of the technical requirements necessary for developing an intelligent, secure, and efficient digital health platform. The project was designed to provide users and administrators with an integrated platform using artificial intelligence, machine learning, predictive analytics, and modern web technologies. The primary objective of the implementation phase was to develop a scalable and user-friendly web application capable of performing disease prediction, safe Over-The-Counter (OTC) medicine recommendations, PDF report generation, geolocation hospital routing, and administrative inventory management effectively.

The system was developed using Python and the Flask framework because of their flexibility, lightweight modularity, and strong support for machine learning and web application development. MongoDB Atlas was used as the cloud database for storing user profiles, symptom logs, disease profiles, medicine inventories, and prediction history records. The implementation focused on building a modular architecture that ensures smooth communication between frontend interfaces, backend services, Scikit-learn machine learning prediction modules, and cloud database collections.

4.1.1 Backend Development

The backend of the platform was developed using Python and the Flask framework. The backend manages all core application functionalities, including user authentication, symptom data processing, Random Forest machine learning inference, Over-The-Counter (OTC) medicine filtering, PDF report generation, and administrative inventory control. MongoDB Atlas was integrated as the cloud database management system for efficient BSON document storage, retrieval, and processing.

Secure authentication and authorization mechanisms were implemented using Werkzeug PBKDF2 password hashing and Flask session management. Role-based access control was incorporated to provide separate access privileges for standard users and system administrators. The backend also handled communication with the Scikit-learn machine learning models and predictive analytics logic for generating disease predictions and verified safe non-prescription medicine recommendations based on user-submitted symptoms.

Machine learning modules were integrated directly into the Flask backend to process health inputs and perform disease prediction analysis using trained feature classification algorithms. The backend coordinated with ReportLab utilities to compile downloadable medical assessment summaries. All processed user information, symptom logs, and prediction results were stored securely within MongoDB collections and utilized for maintaining chronological user history and administrative audit logs.

4.1.2 Frontend Development

The frontend of the platform was designed to provide an interactive, responsive, and user-friendly interface. The user interface enables users and administrators to register, log in, manage profiles, submit health symptoms, access machine learning prediction services, view verified Over-The-Counter (OTC) medicine suggestions, download PDF reports, and monitor system activities seamlessly. The frontend was developed using HTML5, CSS3, and JavaScript with a focus on accessibility, usability, responsiveness, and smooth navigation across desktops, laptops, tablets, and smartphones.

Users can enter health symptoms and parameters through structured web forms before initiating disease prediction processes. Prediction results, confidence margin scores, safe non-prescription medicine recommendations, and nearby hospital routing options via the HTML5 Geolocation API are displayed dynamically to improve user understanding and interaction. Responsive UI elements and clear status indicators enhance communication and provide a seamless triage experience.

The administrative interface was also implemented as part of the frontend, enabling administrators to monitor platform activities, execute CRUD operations on disease and medicine inventories, review prediction audit logs, and coordinate system operations efficiently through centralized SaaS management dashboards.

4.1.3 Integration and Testing

After completing frontend and backend development, both components were integrated to ensure seamless communication and efficient system operation. Extensive testing procedures were conducted to validate the functionality, reliability, security, and performance of the platform. The development process included unit testing, integration testing, white-box testing, black-box testing, security testing, and usability testing to identify and resolve potential system issues.

White-box testing focused on verifying internal application logic, Scikit-learn machine learning feature alignment, Flask backend routes, and database operations. Critical components such as Random Forest disease prediction inference, Over-The-Counter (OTC) safety filtering rules, ReportLab PDF report compilation, and Werkzeug password hashing were thoroughly tested to ensure accuracy, reliability, and stability. Different edge cases, including invalid symptom inputs, concurrent user requests, incomplete form data, and interrupted sessions, were evaluated to optimize system performance and eliminate logical errors.

Black-box testing was conducted from the perspective of end users and administrators to validate system behavior and core functionalities without analyzing internal code structures. Functional testing ensured that major features such as secure user registration, login, symptom submission, AI disease prediction, safe OTC medicine recommendations, PDF report generation, HTML5 geolocation hospital routing, and administrative CRUD dashboards worked correctly. Boundary value analysis and equivalence partitioning techniques were applied to verify that the platform handled different input conditions properly and generated expected outputs.

Usability testing was also performed to evaluate the effectiveness of the web interface and overall user experience. The testing process focused on interface responsiveness, clean

navigation, symptom form usability, report clarity, prediction result presentation, and user interaction throughout platform operations. Minor layout improvements were implemented to enhance accessibility, workflow management, and interaction quality based on testing observations.

The combined results from white-box testing, black-box testing, and security testing confirmed that the **AI-Powered Medical Assistant & OTC Recommendation System** was robust, reliable, secure, and capable of delivering intelligent health triage and non-prescription recommendation services effectively. The successful integration and comprehensive testing process ensured that the platform was ready for deployment and capable of providing efficient services to users and system administrators.

4.2 IMPLEMENTATION RESULTS

The implementation of the **AI-Powered Medical Assistant & OTC Recommendation System** resulted in an intelligent and efficient digital platform that provides users and administrators with a modern health triage and management environment. By utilizing Python, Flask, Scikit-learn machine learning algorithms, predictive analytics, and NoSQL cloud database technologies, the project successfully established seamless communication between frontend web interfaces, backend application services, AI prediction models, and MongoDB Atlas collections. The integration of secure authentication mechanisms (Werkzeug PBKDF2) and role-based access control ensured protected user sessions and reliable system access for all authorized users and administrators.

The platform successfully enabled users to access AI-driven disease prediction services by entering health symptoms through structured web forms according to their requirements. The Random Forest disease prediction engine generated accurate predictions, confidence margin scores, safe Over-The-Counter (OTC) medicine recommendations, and intelligent health insights, improving health awareness and supporting non-prescription guidance.

The implementation of evaluation mechanisms and PDF generation utilities provided accurate health assessments and downloadable reports for users. In addition, the system modules successfully coordinated prediction history tracking, symptom analysis, geolocation hospital routing, and report generation operations, enabling efficient workflow management throughout the platform.

Detailed PDF reports, administrative analytics dashboards, and safety filtering systems helped users and administrators identify health risks, monitor prediction trends, and improve decision-making processes. The OTC medicine filtering module successfully ensured that restricted prescription drugs were excluded, recommending only safe, verified non-prescription remedies based on prediction outcomes.

The admin module effectively facilitated system monitoring, user account management, database CRUD operations on disease profiles and medicine inventories, prediction log reviews, and platform administration through centralized SaaS dashboards. System activity logs and real-time status indicators improved operational oversight by tracking administrative and user actions transparently.

Comprehensive testing and system optimization ensured that the platform delivered stable performance, secure data handling, and responsive operations across multiple devices and concurrent user activities. The modular architecture also positioned the platform for future scalability and integration of advanced technologies such as expanded disease datasets, cloud-based analytics, and enhanced machine learning models.

Overall, the **AI-Powered Medical Assistant & OTC Recommendation System** successfully achieved its objectives of providing an intelligent, secure, scalable, and user-friendly health management environment. The platform effectively combines artificial intelligence, machine learning, predictive analytics, and web technologies to improve health accessibility, support early symptom prediction, enhance safety filtering, and provide efficient digital health services, making it suitable for future expansion and real-world implementation.

CHAPTER 5

CONCLUSION AND FUTURE WORKS

The AI-Powered Medical Assistant & OTC Recommendation System successfully demonstrates the practical application of artificial intelligence, machine learning, and modern web technologies in improving digital health management and intelligent disease prediction services. The project was developed to provide an efficient, secure, and intelligent environment where users and administrators can interact seamlessly while accessing disease prediction, Over-The-Counter (OTC) medicine safety filtering, PDF report generation, geolocation hospital routing, and administrative management functionalities. By integrating Scikit-learn machine learning algorithms, predictive analytics, cloud-based NoSQL database management, and secure authentication mechanisms, the platform provides a comprehensive solution that improves health accessibility, operational efficiency, and non-prescription guidance.

The platform effectively supports AI-based disease prediction, user account management, OTC inventory control, automated PDF reporting, system activity auditing, and real-time web communication. Users are able to input health symptoms to receive intelligent disease predictions, confidence margin scores, and verified safe non-prescription medicine recommendations according to their conditions. The implementation of safety filtering algorithms and structured evaluation mechanisms further enhances the effectiveness of the platform by ensuring safe medication guidance and supporting early symptom identification.

The use of Python and Flask enabled the development of a scalable, reliable, and secure web-based architecture capable of handling multiple operations and concurrent users efficiently. The administrative module provides centralized system monitoring and database management capabilities, ensuring smooth CRUD operations on disease and medicine inventories, activity log tracking, user management, and system administration. Comprehensive testing methodologies confirmed the reliability, usability, security, scalability, and performance of the platform across different functionalities and user interactions.

The project successfully achieved its primary objectives of providing intelligent health management, AI-driven disease prediction, OTC safety filtering, secure user authentication, and efficient report generation. It offers users an accessible and efficient digital platform for monitoring health conditions and accessing guidance while reducing dependency on manual

processes. The system also demonstrates the potential of artificial intelligence and predictive analytics technologies in modern digital health environments.

Overall, the AI-Powered Medical Assistant & OTC Recommendation System serves as an innovative and intelligent solution by combining automation, machine learning, predictive analytics, secure database management, and user-centered web design. The project establishes a strong foundation for future enhancements and advanced technology integration, making it a valuable platform for users, administrators, and digital health service providers.

5.2 FUTURE WORKS

Looking ahead, there are several opportunities for enhancing the functionality, intelligence, scalability, and safety features of the **AI-Powered Medical Assistant & OTC Recommendation System**. Future developments can focus on improving disease prediction accuracy, expanding medical datasets, integrating advanced artificial intelligence frameworks, and providing more personalized health assistance. By incorporating modern machine learning architectures and advanced analytics systems, the platform can further improve management efficiency, health monitoring, symptom prevention, and decision-making capabilities.

Key areas for future enhancement and expansion include:

- **Advanced Machine Learning Models:** Upgrading the current Scikit-learn Random Forest classifier to deep learning architectures or ensemble neural networks to handle more complex symptom combinations and improve overall prediction confidence accuracy.
- **Expanded Disease and Medicine Datasets:** Integrating larger, peer-reviewed medical and pharmacological datasets to broaden the range of detectable health conditions and maintain a more comprehensive inventory of verified Over-The-Counter (OTC) medications and safety flags.
- **Telehealth and Doctor Consultation Integration:** Incorporating secure video consultation features, direct appointment booking with verified medical professionals, and real-time chat modules to bridge the gap between AI triage and professional healthcare providers.
- **Wearable Health Device Integration:** Connecting the platform with APIs from smartwatches and fitness trackers (such as heart rate, blood pressure, and sleep monitors) to ingest real-time physiological parameters automatically for continuous health tracking.
- **Multi language Support:** Implementing regional language support, particularly Malayalam and other Indian languages, to improve accessibility and make health guidance easily understandable for a broader demographic.

5.2.1 Future Implementation Plan

- Integrate advanced machine learning and deep learning models to improve the accuracy and reliability of disease prediction and health analytics across a broader range of medical conditions.

- Enhance analytics modules using advanced predictive techniques and trend analysis to provide more intelligent health insights and early symptom detection capabilities.
- Implement multilingual support to allow users to interact with the platform in multiple languages (such as regional Indian languages), improving accessibility for diverse demographics.
- Develop AI-driven personalized health recommendations based on user history, conditions, lifestyle factors, and prediction outcomes to support preventive health practices and continuous monitoring.
- Integrate telemedicine functionalities such as online consultations, virtual communication, and remote health assistance to improve digital healthcare delivery.
- Implement wearable health device integration to collect real-time data such as heart rate, physical activity, and vital signs for continuous health monitoring and advanced analytics.
- Integrate cloud-based infrastructure and distributed system architecture (MongoDB Atlas scaling, cloud hosting) to improve scalability, reliability, and data management for supporting large-scale operations.
- Develop emergency health alert systems capable of identifying high-risk conditions and notifying emergency contacts or medical services automatically during critical health situations.
- Enhance reporting and analytics modules with advanced data visualizations, predictive dashboards, comparative analysis, and performance tracking features for improved insights.
- Implement electronic health record (EHR) export capabilities to improve record management and support seamless data sharing.
- Develop a native mobile application version of the platform (Android/iOS) using frameworks like Flutter or React Native to improve accessibility and provide flexible management capabilities from mobile devices.
- Integrate AI-powered health chatbots and virtual assistants to provide instant guidance, Over-The-Counter (OTC) safety recommendations, and support for common medical queries.
- Continuously optimize prediction algorithms, safety filtering models, and analytics using user feedback and expanded pharmacological datasets to improve accuracy, efficiency, and user satisfaction.

Overall, the future enhancements of the **AI-Powered Medical Assistant & OTC Recommendation System** will focus on expanding intelligent health capabilities, improving platform accessibility, supporting preventive health practices, and integrating advanced digital technologies to create a more comprehensive, scalable, and intelligent health management environment.

REFERENCES

Conference Proceedings:

1. Shickel, B., Tighe, P. J., Bihorac, A., & Rashidi, P. (2018, July). Deep EHR: A Survey of Recent Advances in Deep Learning Techniques for Electronic Health Record Analysis. *IEEE Journal of Biomedical and Health Informatics*, pp. 1589-1604. IEEE.
2. Ramesh, A. N., Kambhampati, C., Monson, J. R., & Drew, P. J. (2020, March). Artificial Intelligence in Medicine. *Annals of The Royal College of Surgeons of England*, pp. 188-194. Royal College of Surgeons.

Journal Articles:

1. Rajkomar, A., Dean, J., & Kohane, I. (2019). Machine Learning in Medicine. *New England Journal of Medicine*, 380(14), 1347-1358.
2. Topol, E. (2019). High-performance Medicine: The Convergence of Human and Artificial Intelligence. *Nature Medicine*, 25(1), 44-56.
3. Esteva, A., Robicquet, A., Ramsundar, B., Kuleshov, V., DePristo, M., Chou, K., ... & Dean, J. (2019). A Guide to Deep Learning in Healthcare. *Nature Medicine*, 25(1), 24-29.
4. Jiang, F., Jiang, Y., Zhi, H., Dong, Y., Li, H., Ma, S., ... & Wang, Y. (2020). Artificial Intelligence in Healthcare: Past, Present and Future. *Stroke and Vascular Neurology*, 2(4), 230-243.
5. Miotto, R., Wang, F., Wang, S., Jiang, X., & Dudley, J. T. (2018). Deep Learning for Healthcare: Review, Opportunities and Challenges. *Briefings in Bioinformatics*, 19(6), 1236-1246.

APPENDIXES

Appendix A- Screenshots

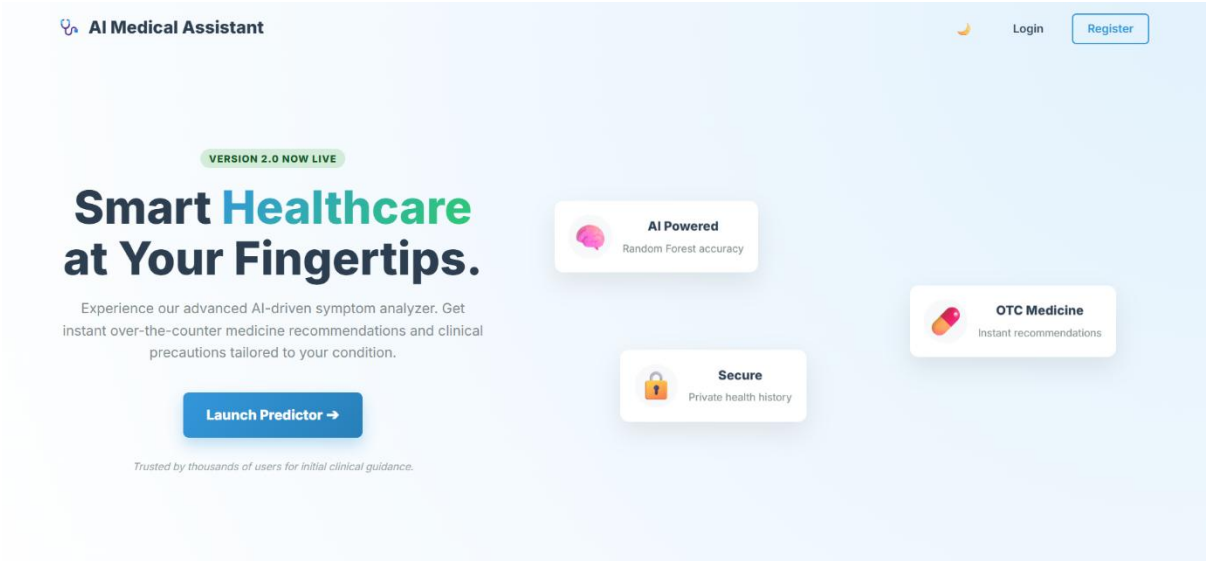


Figure A.1 Home Page

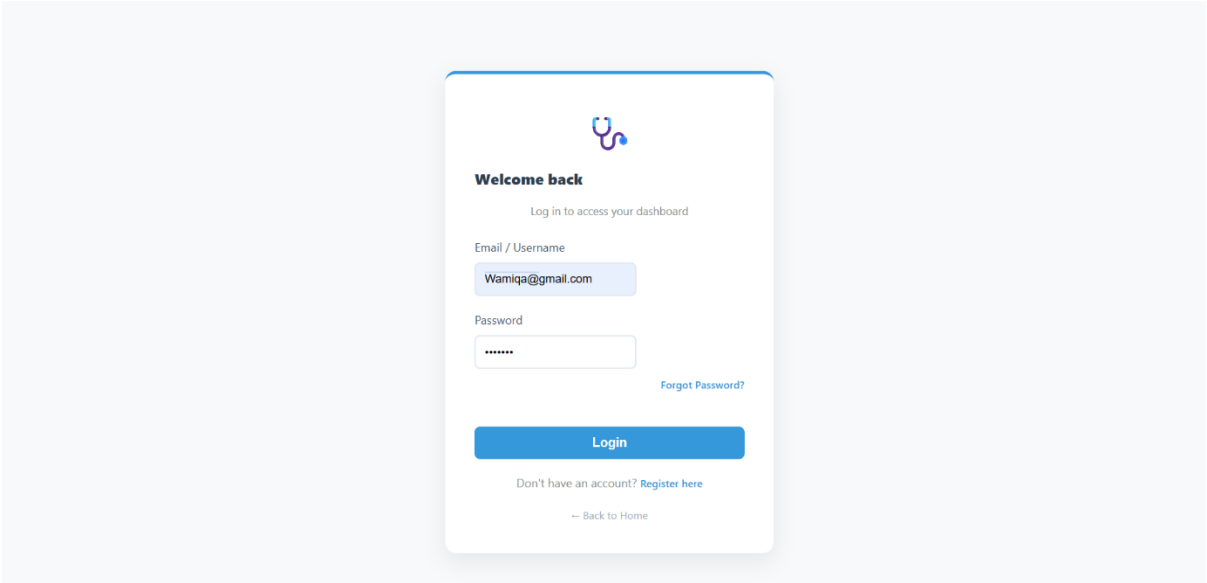


Figure A.2 Login Page

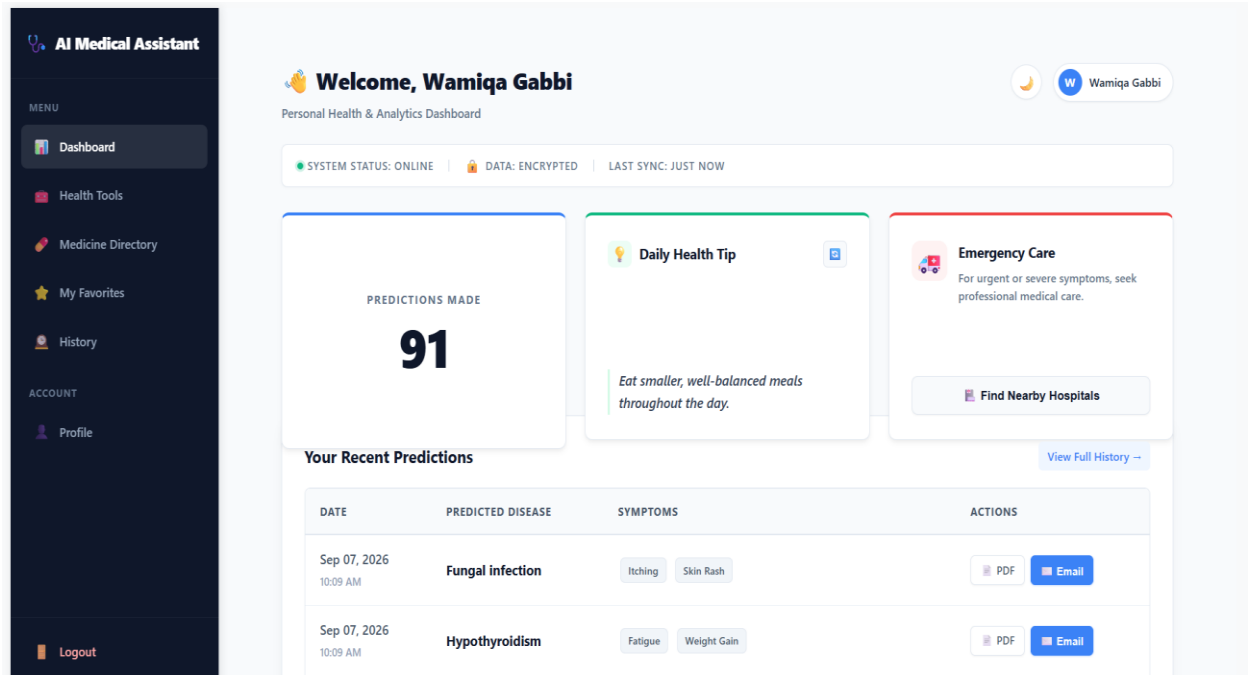


Figure A.3: User Dashboard

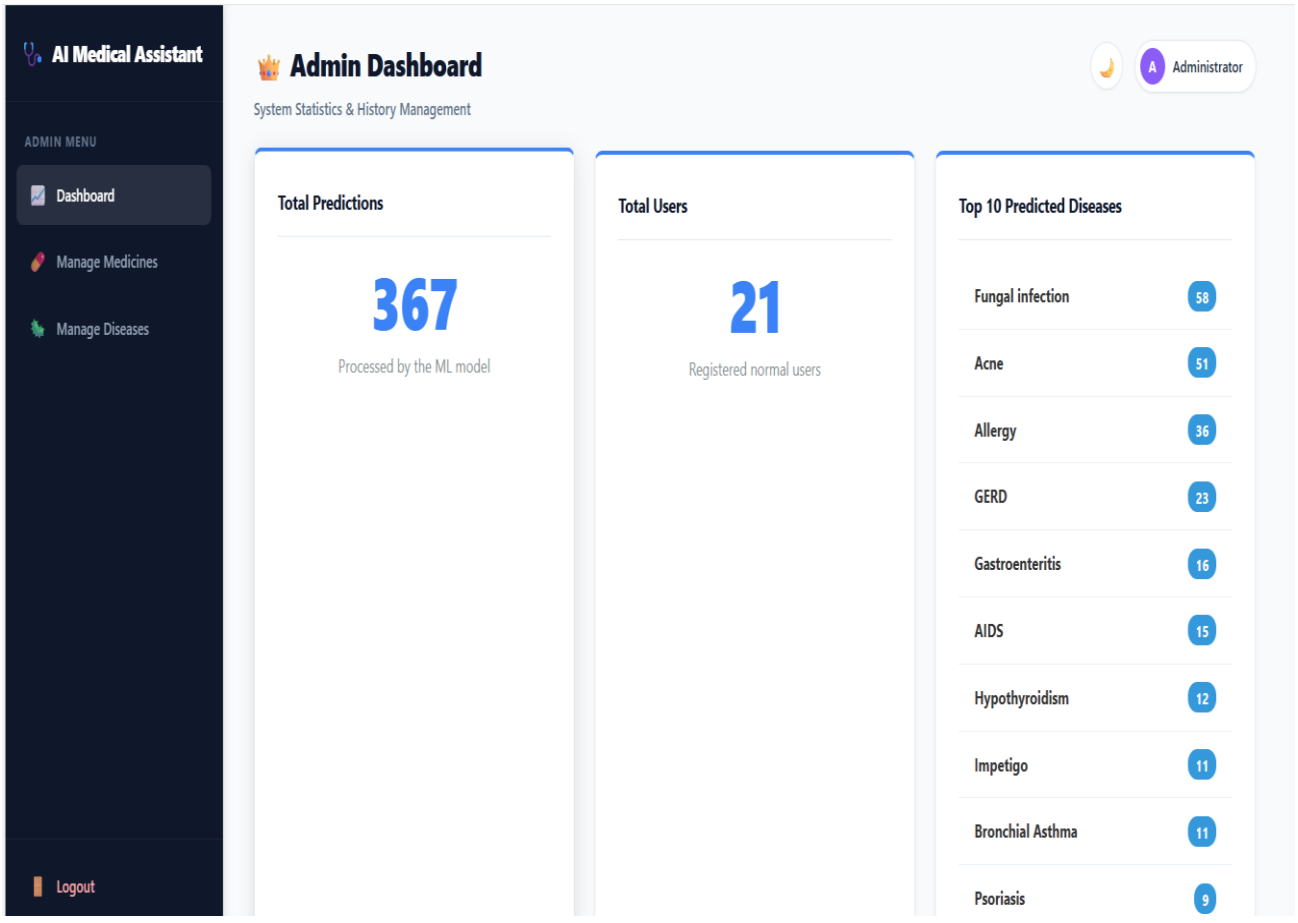


Figure A.3: Admin Dashboard

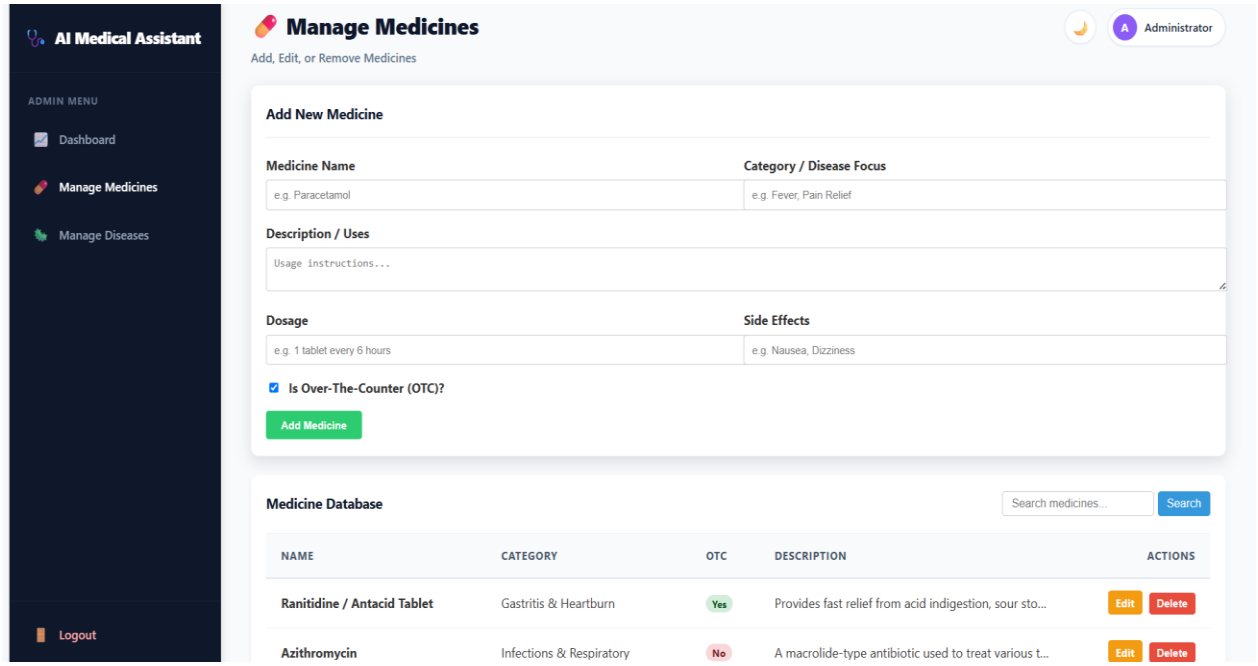


Figure A.3: Admin Medicine page

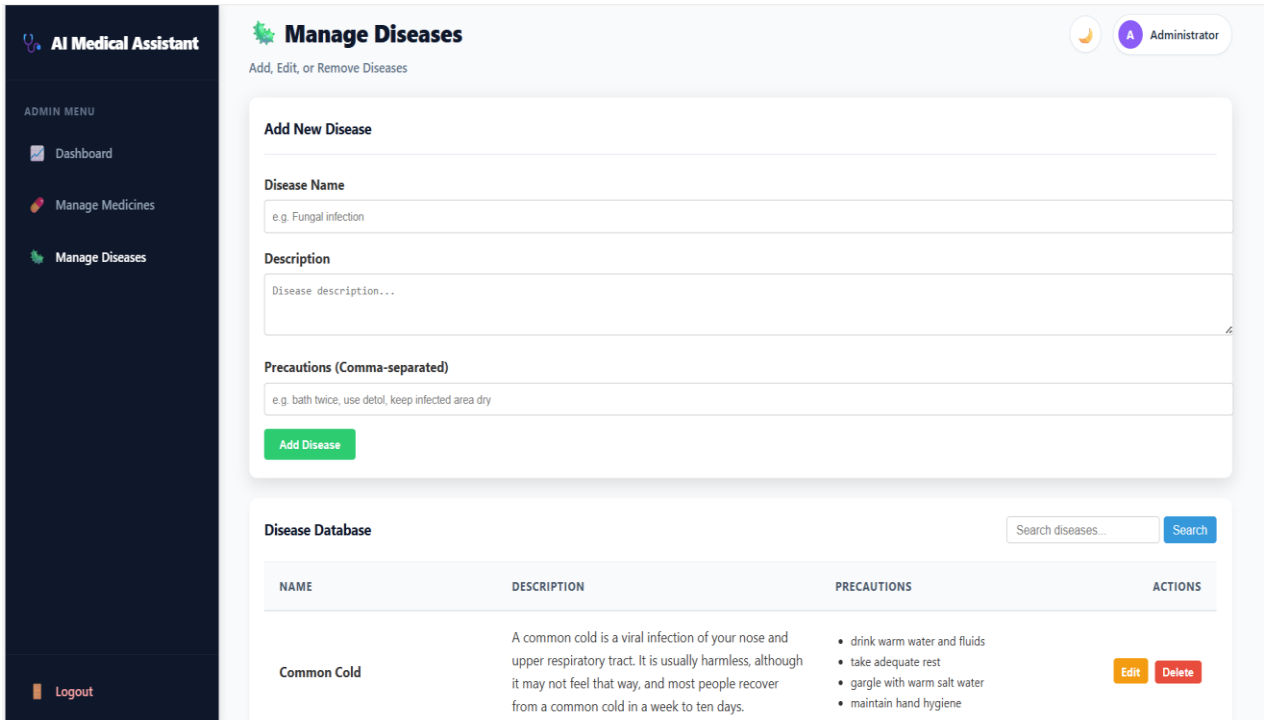


Figure A.3: Admin Disease page

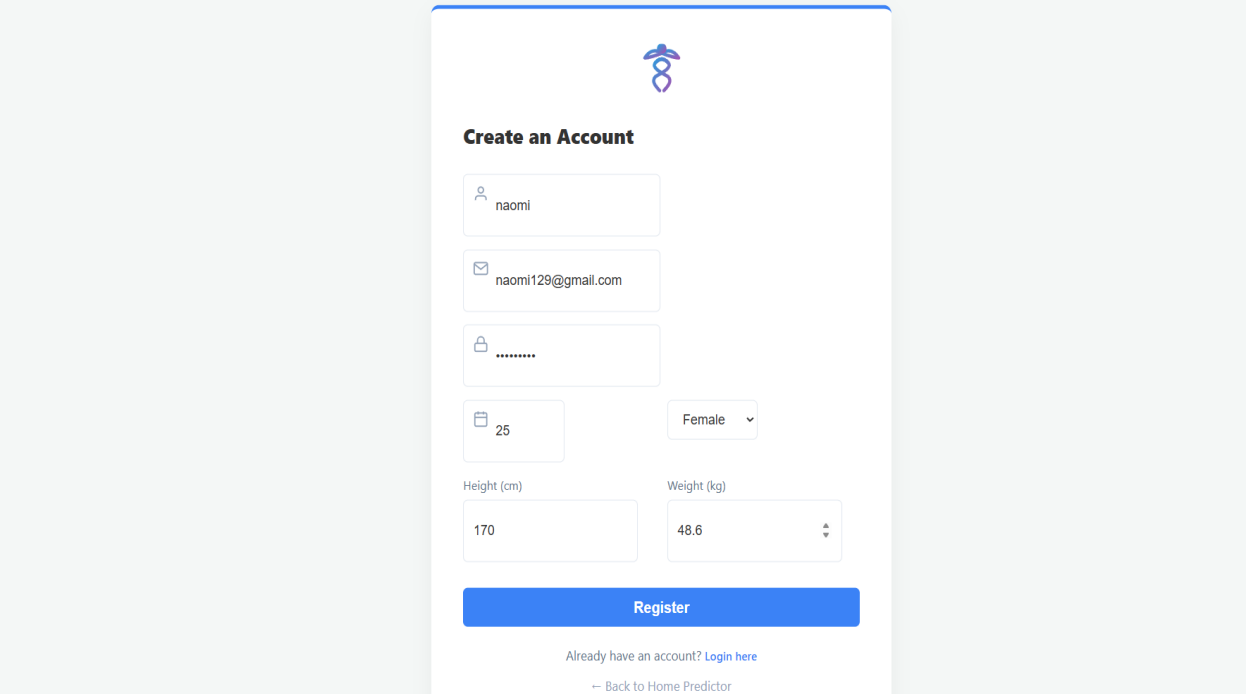


Figure A.3: Register page

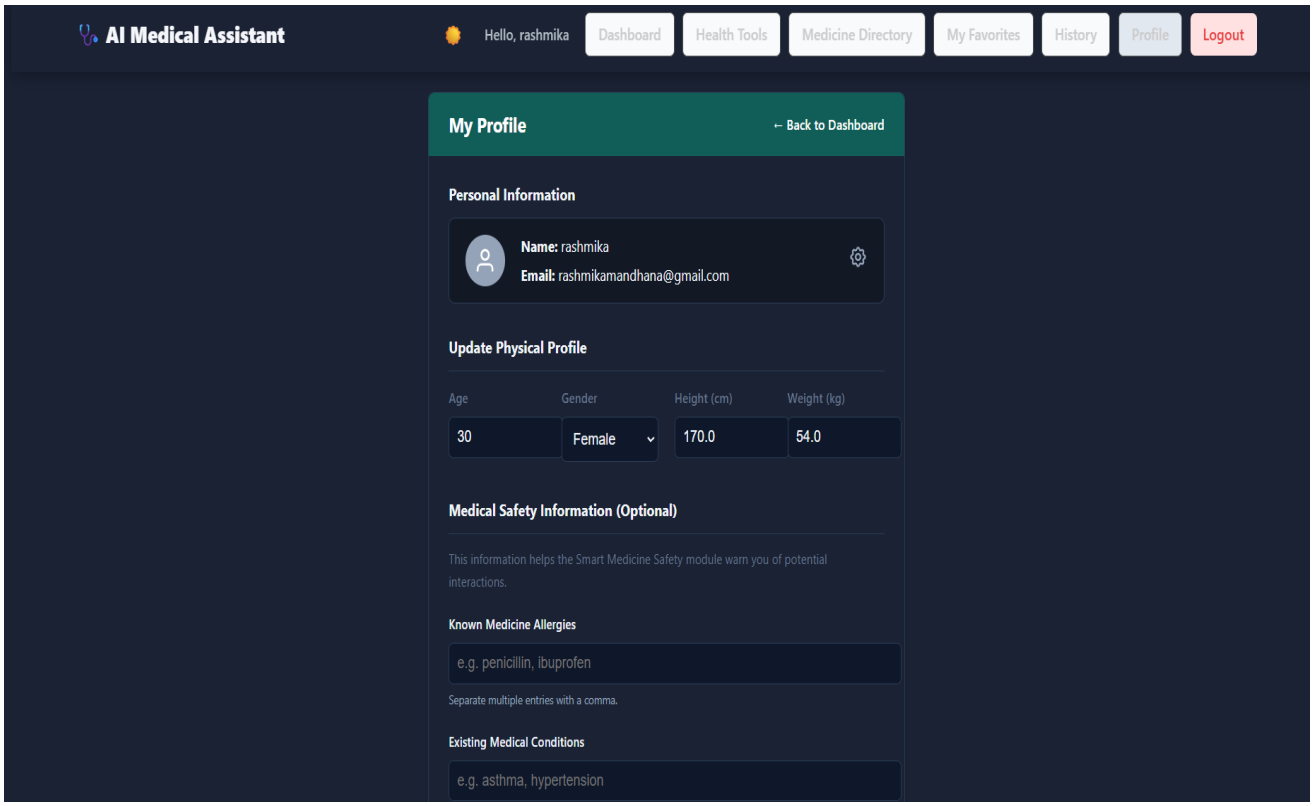


Figure A.3: User Profile page

Appendix B- Sample Code

App.py/

```
import os
import json
import pandas as pd
from flask import Flask, request, jsonify, render_template, session, redirect, url_for, flash
from flask_cors import CORS
import numpy as np
import joblib
from pymongo import MongoClient
from datetime import datetime
import logging
from dotenv import load_dotenv

load_dotenv()
from werkzeug.security import check_password_hash
from flask_mail import Mail

app = Flask(__name__)

# --- Flask-Mail Configuration ---
app.config['MAIL_SERVER'] = os.environ.get('MAIL_SERVER', 'smtp.gmail.com')
app.config['MAIL_PORT'] = int(os.environ.get('MAIL_PORT', 587))
app.config['MAIL_USE_TLS'] = os.environ.get('MAIL_USE_TLS', 'True').lower() in ['true',
'1', 't']
```

```
app.config['MAIL_USERNAME'] = os.environ.get('MAIL_USERNAME')
app.config['MAIL_PASSWORD'] = os.environ.get('MAIL_PASSWORD')
app.config['MAIL_DEFAULT_SENDER'] = os.environ.get('MAIL_DEFAULT_SENDER')
mail = Mail(app)

# --- Configure Logger ---
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
)
logger = logging.getLogger("MedicalApp")

# --- MongoDB Configuration (Secure .env with Fallback) ---

db = None
prediction_history = None

try:
    client = MongoClient(MONGO_URI)
    db = client.get_database("medicine_recommendation_db")
    prediction_history = db.prediction_history
    logger.info("Connected to MongoDB Atlas successfully!")
except Exception as e:
    logger.error(f"Failed to connect to MongoDB: {e}")
```

```
# --- App Security & Blueprints ---
app.secret_key = os.environ.get("FLASK_SECRET_KEY", "super-secret-admin-key-replace-in-prod")
CORS(app)

from admin import admin_bp
app.register_blueprint(admin_bp, url_prefix='/admin')

from user.routes import user_bp
app.register_blueprint(user_bp, url_prefix='/user')

# --- Load all the pickled files using joblib ---
rf_model = None
feature_columns = None
description_dict = None
precautions_dict = None
medicine_dict = None
disease_classes = None

try:
    logger.info("Loading models and datasets...")

    BASE_DIR = os.path.dirname(os.path.abspath(__file__))
    MODELS_DIR = os.path.join(BASE_DIR, 'models')
```

```
    rf_model = joblib.load(os.path.join(MODELS_DIR, 'rf_model.pkl'))
    feature_columns = list(joblib.load(os.path.join(MODELS_DIR, 'feature_columns.pkl')))
    description_dict = joblib.load(os.path.join(MODELS_DIR, 'description.pkl'))
    precautions_dict = joblib.load(os.path.join(MODELS_DIR, 'precaution.pkl'))

    med_df = joblib.load(os.path.join(MODELS_DIR, 'medicine.pkl'))
    medicine_dict = dict(zip(med_df['Disease'], med_df['Medication']))

    disease_classes = sorted(list(description_dict.keys()))

    logger.info("All files loaded successfully!")
except Exception as e:
    logger.critical(f"Critical error loading files: {e}", exc_info=True)

# Basic Home Route (Landing Page)
@app.route('/')
def home():
    return render_template('landing.html')

# Unified Login System
@app.route('/login', methods=['GET', 'POST'])
def login():
    if session.get('user_id'):
```

```
    return redirect(url_for('home'))
if session.get('is_admin'):
    return redirect(url_for('admin.dashboard'))

error = None
if request.method == 'POST':
    email_or_username = request.form.get('email', '').strip()
    password = request.form.get('password', '')

    # 1. Admin Authentication Check
    ADMIN_USER = os.environ.get('ADMIN_USERNAME')
    ADMIN_PASS = os.environ.get('ADMIN_PASSWORD')

    if ADMIN_USER and email_or_username == ADMIN_USER:
        if password == ADMIN_PASS:
            session.clear() # Securely clear all other session data
            session['is_admin'] = True
            flash("Admin login successful", "success")
            return redirect(url_for('admin.dashboard'))
        else:
            flash('Invalid admin credentials', 'error')
            return redirect(url_for('login'))

    # 2. Normal User Authentication Check
```

```
        if db is None:
                return render_template('login.html', error="Database connection is currently unavailable. Please try again later.")

        users_collection = db.users
        user = users_collection.find_one({"email": email_or_username})

        if user and check_password_hash(user['password'], password):
            session.pop('admin_logged_in', None)
            session['user_id'] = str(user['_id'])
            session['user_name'] = user['full_name']
            next_page = request.args.get('next')
            return redirect(next_page or url_for('home'))

        # 3. Invalid credentials
        error = "Invalid email/username or password."

    return render_template('login.html', error=error)

@app.route('/logout')
def logout():
    session.pop('user_id', None)
    session.pop('user_name', None)
    session.pop('is_admin', None)
    return redirect(url_for('home'))
```

```
# Predictor UI Route
from user.auth import user_required

@app.route('/predictor')
@user_required
def predictor():
    return render_template('index.html')

# --- History Route (Optimized with limit) ---

@app.route('/history')
@user_required
def history():
    from flask import session, request
    from datetime import datetime, timedelta
    try:
        user_id = session.get('user_id')
        query = {"user_id": user_id}

        search = request.args.get('search')
        date_filter = request.args.get('date_filter')

        if search:
```

```
            query["predicted_disease"] = {"$regex": search, "$options": "i"}

        if date_filter:
            now = datetime.now()
            if date_filter == 'today':
                start_date = now.replace(hour=0, minute=0, second=0, microsecond=0)
                query["timestamp"] = {"$gte": start_date}
            elif date_filter == 'week':
                start_date = now - timedelta(days=now.weekday())
                start_date = start_date.replace(hour=0, minute=0, second=0, microsecond=0)
                query["timestamp"] = {"$gte": start_date}
            elif date_filter == 'month':
                start_date = now.replace(day=1, hour=0, minute=0, second=0, microsecond=0)
                query["timestamp"] = {"$gte": start_date}

        records = list(prediction_history.find(query).sort('_id', -1).limit(50))
                    return render_template('history.html', records=records, search=search,
date_filter=date_filter)
    except Exception as e:
        logger.error(f"Error loading history: {e}")
        return render_template('500.html'), 500

@app.route('/predict', methods=['POST'])
def predict_disease():
    try:
```

```
    if rf_model is None or not feature_columns:
        return jsonify({"error": "The prediction models are currently unavailable. Please try again later."}), 503

    if not request.is_json:
        logger.warning("Prediction rejected: Request payload was not JSON.")
        return jsonify({"error": "Request must be JSON."}), 400

    data = request.get_json()
    if not data or 'symptoms' not in data:
        logger.warning("Prediction rejected: Missing 'symptoms' in JSON payload.")
        return jsonify({"error": "Missing 'symptoms' in JSON payload."}), 400

    user_symptoms = data.get('symptoms')

    if not isinstance(user_symptoms, list):
        logger.warning("Prediction rejected: 'symptoms' field was not a list.")
        return jsonify({"error": "'symptoms' must be a list."}), 400

    if len(user_symptoms) == 0:
        logger.warning("Prediction rejected: Symptom list was empty.")
        return jsonify({"error": "Symptom list cannot be empty."}), 400

    valid_symptoms = [symp for symp in user_symptoms if isinstance(symp, str)]
```

```
    if len(valid_symptoms) == 0:
        logger.warning("Prediction rejected: No valid/recognized symptoms provided.")
        return jsonify({"error": "None of the provided symptoms are valid/recognized."}), 400

    logger.info(f"Received prediction request with {len(valid_symptoms)} valid symptoms.")

    input_vector = np.zeros(len(feature_columns))

    # Updated flexible mapping across all 17 symptom positions to prevent wrong/default predictions
    for symptom in valid_symptoms:
        clean_sym = symptom.strip()
        for i in range(1, 18):
            feature_name = f"Symptom_{i}_{clean_sym}"
            feature_name_spaced = f"Symptom_{i}_ {clean_sym}"

            if feature_name in feature_columns:
                idx = feature_columns.index(feature_name)
                input_vector[idx] = 1
            elif feature_name_spaced in feature_columns:
                idx = feature_columns.index(feature_name_spaced)
                input_vector[idx] = 1

    disease_raw = str(rf_model.predict([input_vector])[0])
```

```
        try:
            probabilities = rf_model.predict_proba([input_vector])[0]
            # Academic normalization: Because probabilities are distributed across 40+ classes,
             # an absolute probability of 0.35 can be overwhelmingly decisive if the next best is 0.02.
            # We calculate "Relative Confidence Margin" against the closest competing disease.
            sorted_probs = sorted(probabilities, reverse=True)
            max_prob = sorted_probs[0]
            second_prob = sorted_probs[1] if len(sorted_probs) > 1 else 0

            if (max_prob + second_prob) > 0:
                relative_confidence = max_prob / (max_prob + second_prob)
            else:
                relative_confidence = max_prob

            confidence_score = int(round(relative_confidence * 100))

            # Floor it to the absolute probability so it never decreases, and cap at 98%
            confidence_score = min(98, max(confidence_score, int(round(max_prob * 100))))

        except AttributeError:
            # Fallback if predict_proba is not available
            confidence_score = 95

        # 1. Fix disease-name typo in user-facing output
```

```
canonical_name_map = {
    "Peptic ulcer diseae": "Peptic ulcer disease",
    "Dimorphic hemmorhoids(piles)": "Dimorphic hemorrhoids(piles)",
    "(vertigo) Paroymsal  Positional Vertigo": "(vertigo) Paroymsal Positional Vertigo"
}
disease = canonical_name_map.get(disease_raw, disease_raw)

def get_from_dict(d, search_key, default_val):
    import re
    def normalize(text):
        return re.sub(r'\s+', ' ', str(text).lower().strip())

    normalized_search = normalize(search_key)

    # Direct match
    for k, v in d.items():
        if normalize(k) == normalized_search:
            return v

    # Known aliases across datasets
    aliases = [
        "peptic ulcer diseae", "peptic ulcer disease",
        "dimorphic hemorrhoids(piles)", "dimorphic hemmorhoids(piles)",
        "(vertigo) paroymsal positional vertigo", "(vertigo) paroymsal  positional vertigo"
```

```
    ]
    if normalized_search in aliases:
        for k, v in d.items():
            if normalize(k) in aliases:
                return v
    return default_val

description = str(get_from_dict(description_dict, disease, "Description not available."))

# Helper to clean and parse string representations of lists from datasets
def parse_raw_list(raw_data, fallback):
    import ast
    if isinstance(raw_data, str):
        try:
            parsed = ast.literal_eval(raw_data)
            return parsed if isinstance(parsed, list) else [str(raw_data)]
        except:
            return [x.strip() for x in raw_data.split(',')]
    elif isinstance(raw_data, (list, np.ndarray)):
        return [str(x) for x in raw_data if str(x) != 'nan' and x]
    return fallback

raw_precautions = get_from_dict(precautions_dict, disease, ["No precautions found."])
precautions = parse_raw_list(raw_precautions, [str(raw_precautions)])
```

```
    raw_medicines = get_from_dict(medicine_dict, disease, [])
    medications = parse_raw_list(raw_medicines, [])

    # Clean any remaining brackets or quotes
    medications = [str(m).strip(" '\"[]") for m in medications if str(m).strip(" '\"[]")]
    precautions = [str(p).strip(" '\"[]") for p in precautions if str(p).strip(" '\"[]")]

    # STRICT OTC FILTERING: Only verified OTC medicines from db.medicines
    verified_otc_meds = []
    if db is not None:
        import re
        for m in medications:
            med_doc = db.medicines.find_one({"name": {"$regex": f"^{re.escape(m)}$", "$options": "i"}, "is_otc": True})
            if med_doc:
                verified_otc_meds.append(m)
    medications = verified_otc_meds

    if not medications:
        medications = ["No suitable OTC medicine available. Please consult a qualified healthcare professional."]
    if not precautions:
        precautions = ["No precautions found."]
```

```
try:
    from flask import session
    history_record = {
        "selected_symptoms": user_symptoms,
        "predicted_disease": disease,
        "medicines": medications,
        "precautions": precautions,
        "timestamp": datetime.now()
    }

    if session.get('user_id'):
        user_id = session.get('user_id')
        history_record['user_id'] = user_id

        # Auto-save recommended medicines to favorites
        try:
            import re
            favorites_collection = db.favorites
            medicines_collection = db.medicines

            for med_name in medications:
                if "No suitable OTC medicine available" in med_name:
                    continue
```

```
                med_name_clean = str(med_name).strip(" '\"[]")
                if not med_name_clean:
                    continue

                        med_doc = medicines_collection.find_one({"name": {"$regex": f"^{re.escape(med_name_clean)}$", "$options": "i"}})
                if med_doc:
                    med_id_str = str(med_doc['_id'])
                    existing = favorites_collection.find_one({"user_id": user_id, "medicine_id": med_id_str})
                    if not existing:
                        favorites_collection.insert_one({
                            "user_id": user_id,
                            "medicine_id": med_id_str,
                            "created_at": datetime.utcnow()
                        })
            except Exception as fav_err:
                logger.error(f"Error auto-saving favorites: {fav_err}")

        prediction_history.insert_one(history_record)
    except Exception as e:
        logger.error(f"Error saving history to MongoDB: {e}")

    logger.info(f"Prediction successful. Result: {disease}")
```

```
        return jsonify({
            "disease": disease,
            "description": description,
            "precautions": precautions,
            "medications": medications,
            "confidence_score": confidence_score,
           "disclaimer": "This data is for educational purposes only. Listed treatments often require prescriptions. Do not self-medicate; always consult a doctor."
        })
    except Exception as e:
        logger.error(f"Internal Prediction Error: {str(e)}", exc_info=True)
         return jsonify({"error": "An unexpected error occurred while processing your request. Please try again."}), 500

# --- Custom Error Handlers ---
@app.errorhandler(404)
def not_found_error(error):
    return render_template('404.html'), 404

@app.errorhandler(500)
def internal_error(error):
    return render_template('500.html'), 500

if __name__ == '__main__':
    app.run(debug=True, port=5000)
```